Manipulación y consulta de datos con SQL

Breve descripción:

Este componente formativo desarrolla competencias en lenguaje de manipulación de datos (DML) para bases de datos relacionales. Aborda inserción, consulta, modificación y eliminación de datos, además de JOIN, subconsultas, funciones, procedimientos almacenados y almacenamiento en la nube. Su enfoque fortalece la capacidad para desarrollar soluciones eficientes y correctas de acceso a datos en entornos profesionales reales.

Octubre 2026

Tabla de contenido

Introducción 4

1. Fundamentos del lenguaje de manipulación de datos (DML) 6

1.1. Comandos de inserción, modificación y eliminación de registros 8

1.2. Búsqueda y recuperación de datos con el comando SELECT 18

2. Sintaxis y cláusulas del comando SELECT 24

2.1. Cláusulas de selección y filtrado de datos 24

2.2. Agrupación, ordenamiento y condiciones sobre grupos 32

3. Funciones para la manipulación y análisis de datos 40

3.1. Funciones de cadena 41

3.2. Funciones de agregación 49

4. Consultas avanzadas en SQL 54

4.1. Subconsultas y tipos de subconsultas 55

4.2. Consultas combinadas y tipos de JOIN 62

5. Procedimientos almacenados en bases de datos 71

5.1. Creación y definición de procedimientos almacenados 72

5.2. Parámetros y ejecución de procedimientos 81

6. Escenarios de almacenamiento y bases de datos en la nube 87

6.1. Soluciones de almacenamiento en la nube y APIs multimodelo 88

6.2. Tipos de bases de datos en entornos cloud 91

Síntesis 96

Glosario 98

Referencias bibliográficas 101

Créditos 102

Introducción

Las bases de datos relacionales constituyen el pilar tecnológico sobre el cual se sustenta la gran mayoría de los sistemas de información empresariales, gubernamentales y académicos del mundo contemporáneo. Detrás de cada transacción bancaria, cada compra en línea, cada consulta médica registrada en un sistema hospitalario y cada reporte gerencial generado en tiempo real, existe una base de datos relacional que almacena, organiza y entrega los datos con precisión y eficiencia. El lenguaje que hace posible toda esta operación es SQL —Structured Query Language—, el estándar universal para la manipulación y consulta de datos en sistemas relacionales, definido por la norma ISO/IEC 9075 y adoptado por todos los principales sistemas manejadores de bases de datos del mercado.

SQL no es simplemente un lenguaje técnico para programadores:

Es el vocabulario común que permite a analistas, desarrolladores, administradores de bases de datos y profesionales de negocio comunicarse con las estructuras de datos de las organizaciones.

A diferencia de los lenguajes de programación de propósito general como Java o Python, SQL está diseñado específicamente para expresar operaciones sobre datos de manera declarativa:

El programador describe qué resultado desea obtener, y el motor de base de datos decide cómo obtenerlo de la manera más eficiente posible, basándose en estadísticas y planes de ejecución optimizados.

Esta separación entre el qué y el cómo es uno de los principios más poderosos del modelo relacional.

Dominar SQL es, en el contexto profesional actual, una competencia tan fundamental como saber leer y escribir en el mundo del trabajo con datos. Según el índice TIOBE de popularidad de lenguajes de programación y las encuestas anuales de Stack Overflow, SQL se mantiene consistentemente entre los cinco lenguajes más utilizados por los profesionales de tecnología, por encima de muchos lenguajes de programación modernos.

Este componente formativo aborda el lenguaje SQL desde su dimensión más operativa y práctica: el lenguaje de manipulación de datos (DML), que comprende todas las operaciones para insertar, modificar, eliminar y consultar datos en una base de datos. A lo largo de seis temas estructurados de manera progresiva, se transitará desde los comandos fundamentales de inserción, modificación y eliminación, pasando por la sintaxis completa del comando SELECT con todas sus cláusulas, las funciones de cadena y agregación, las subconsultas, los diferentes tipos de JOIN para combinar tablas, los procedimientos almacenados, hasta las soluciones modernas de almacenamiento en la nube con sus múltiples paradigmas de bases de datos.

Cada tema incluye ejemplos de código SQL reales, comentados y progresivos en complejidad, tablas de referencia rápida con la sintaxis de los comandos más utilizados, así como comparativas que facilitan la comprensión de las diferencias entre las distintas opciones disponibles. El aprendizaje de SQL es eminentemente práctico: los conceptos solo se consolidan completamente cuando se escriben y ejecutan sentencias reales contra bases de datos reales, verificando en tiempo real el comportamiento de cada sentencia.

1. Fundamentos del lenguaje de manipulación de datos (DML)

El lenguaje de manipulación de datos, conocido por sus siglas en inglés DML (Data Manipulation Language), es el subconjunto del lenguaje SQL que permite a los usuarios y aplicaciones interactuar con los datos almacenados en una base de datos relacional. A diferencia del Lenguaje de Definición de Datos (DDL), que se ocupa de crear y modificar la estructura de las tablas (mediante comandos como CREATE TABLE, ALTER TABLE o DROP TABLE), el DML se enfoca exclusivamente en los datos que residen dentro de esas estructuras: permite agregar nuevos registros, actualizar los existentes, eliminar los obsoletos y, sobre todo, consultar y recuperar información de manera precisa y eficiente.

El DML es el lenguaje del día a día de cualquier aplicación de software que trabaje con bases de datos. Cada vez que un usuario registra una venta en un sistema de punto de venta, actualiza su perfil en una red social, cancela un pedido en una tienda en línea o genera un reporte de ventas del mes anterior para la gerencia, está —de manera transparente, sin ser consciente de ello— ejecutando sentencias DML contra la base de datos subyacente. Para el desarrollador de software, comprender profundamente el DML no es solo una cuestión técnica: es comprender cómo el software que construye interactúa con los datos del negocio, qué impacto tienen sus sentencias sobre el rendimiento del sistema y cómo garantizar la integridad y consistencia de la información en todo momento, incluso cuando múltiples usuarios acceden simultáneamente a los mismos datos.

El lenguaje SQL, en su conjunto, se divide en varios sublenguajes especializados según el tipo de operación que realizan:

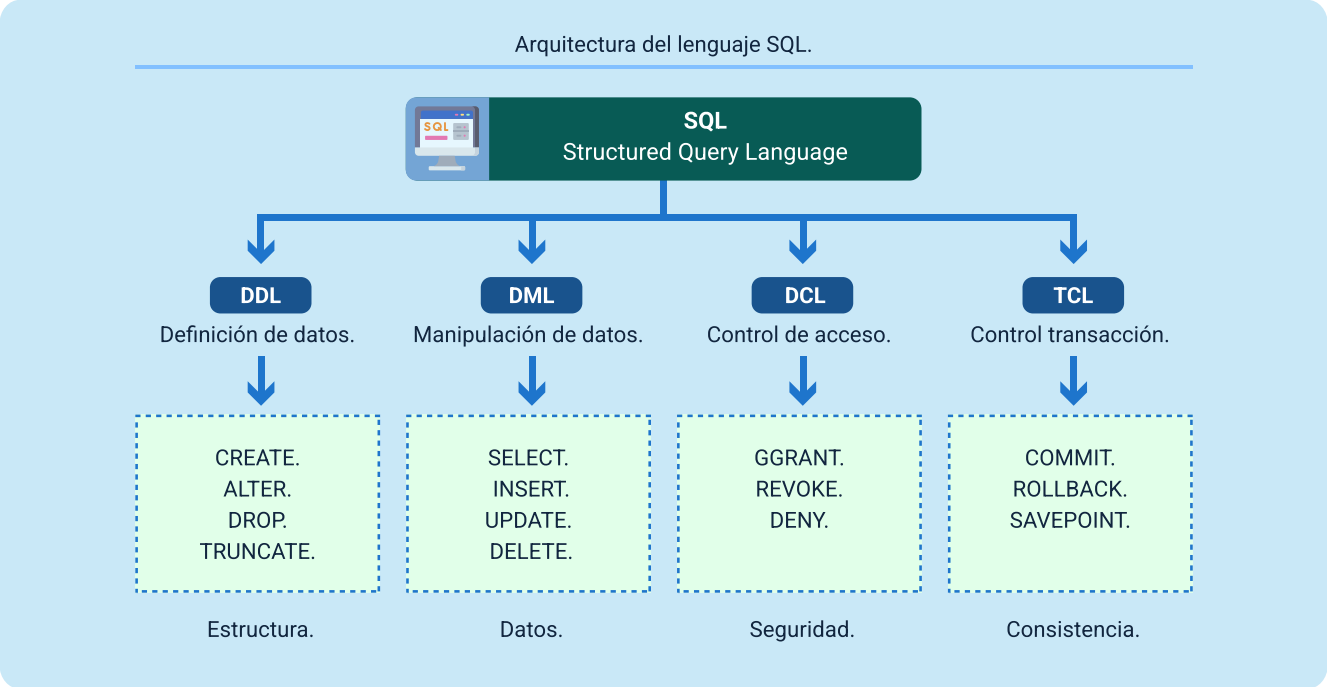
a) **DDL (Data Definition Language)**: se ocupa de definir y modificar la estructura del esquema de la base de datos.
b) **DCL (Data Control Language)**: gestiona los permisos de acceso y seguridad.
c) **TCL (Transaction Control Language)**: controla las transacciones mediante los comandos COMMIT, ROLLBACK y SAVEPOINT.
d) **DML (Data Manipulation Language)**: objeto de estudio de este componente, se ocupa de todas las operaciones sobre los datos.

Es importante comprender esta arquitectura del lenguaje para entender qué tipo de operación se está realizando en cada momento y qué consecuencias tiene sobre el estado de la base de datos.

Los cuatro comandos fundamentales del DML son: INSERT (para insertar nuevos registros en una tabla), UPDATE (para modificar los valores de registros existentes), DELETE (para eliminar registros que ya no son necesarios) y SELECT (para consultar y recuperar datos de una o más tablas). De estos cuatro, SELECT es con mucha diferencia el más complejo, el más utilizado y el más poderoso, y merece —como se indicará en los temas siguientes— varias temáticas dedicadas exclusivamente a sus múltiples cláusulas, variaciones y combinaciones. Los tres primeros (INSERT, UPDATE, DELETE) conforman el grupo de comandos de escritura o modificación de datos, que son los que alteran el estado persistente de la base de datos y, por tanto, requieren una comprensión precisa de sus efectos, de las precauciones necesarias para utilizarlos correctamente y de los mecanismos de control de transacciones que protegen la integridad de los datos ante fallos del sistema.

La siguiente imagen ejemplifica la arquitectura de este lenguaje:

Figura 1. Arquitectura del lenguaje SQL y sus sublenguajes principales



1.1. Comandos de inserción, modificación y eliminación de registros

Los comandos de modificación del DML —INSERT, UPDATE y DELETE— son las operaciones que alteran el estado persistente de la base de datos. Cada vez que se ejecuta uno de estos comandos, el SMBD registra el cambio de manera permanente en el almacenamiento físico, a menos que la operación esté enmarcada dentro de una transacción que posteriormente se revierta (ROLLBACK). Por ello, su uso requiere comprensión clara de las tablas afectadas, las condiciones de filtrado que determinan qué registros se modifican o eliminan y las implicaciones para la integridad referencial del esquema relacional.

Un principio fundamental que todo desarrollador debe interiorizar al trabajar con los comandos de modificación del DML es el principio de mínima sorpresa: antes de

ejecutar cualquier UPDATE o DELETE sobre una base de datos de producción, debe ejecutarse primero el SELECT correspondiente con la misma cláusula WHERE para verificar que los registros que se van a modificar o eliminar son exactamente los que se intenta afectar. Esta práctica preventiva, aunque sencilla, evita errores catastróficos que podrían resultar en pérdida irreversible de datos.

A. Comando INSERT—Inserción de registros

El comando INSERT permite agregar uno o más registros nuevos a una tabla de la base de datos. Es el punto de entrada de los datos al sistema y su correcta implementación es fundamental para garantizar la integridad y calidad de la información almacenada. Existen varias formas de utilizar el INSERT, cada una adecuada para diferentes escenarios y necesidades.

La forma más común y recomendada es la forma explícita, donde se especifican los nombres de las columnas en las que se van a insertar los datos, seguidos de los valores correspondientes. Esta forma es preferible a la forma implícita (sin especificar nombres de columnas) porque hace el código más legible, más resistente a cambios en la estructura de la tabla y reduce significativamente el riesgo de insertar datos en columnas incorrectas. Si en el futuro se agrega una nueva columna a la tabla, las sentencias INSERT explícitas continuarán funcionando correctamente, mientras que las implícitas podrían fallar o producir resultados incorrectos.

Otra variante importante es el INSERT múltiple, que permite insertar varios registros en una sola sentencia. Esta forma es significativamente más eficiente que ejecutar múltiples INSERT individuales, porque reduce el número de viajes de red entre la aplicación y el servidor de base de datos, y permite al SMBD optimizar la operación

de escritura de múltiples registros en una sola transacción. En entornos de carga masiva de datos, esta diferencia de rendimiento puede ser de varios órdenes de magnitud.

La variante INSERT...SELECT es especialmente poderosa porque permite insertar el resultado de una consulta compleja directamente en una tabla, sin necesidad de recuperar los datos en la aplicación y re-enviarlos al servidor. Esto es fundamental para operaciones de transformación de datos, carga de tablas de resumen o migración de datos entre tablas del mismo esquema o de esquemas diferentes.

Ejemplo:

```
-- Forma explícita (recomendada siempre)
INSERT INTO clientes (id_cliente, nombre, email, telefono, ciudad, fecha_registro)
VALUES (1, 'Ana García', 'ana@email.com', '3001234567', 'Bogotá', GETDATE());
-- Inserción múltiple en una sola sentencia (más eficiente)
INSERT INTO clientes (id_cliente, nombre, email, telefono, ciudad)
VALUES
 (2, 'Carlos López',  'carlos@email.com', '3109876543', 'Medellín'),
 (3, 'María Torres',  'maria@email.com',  '3204567890', 'Cali'),
 (4, 'Pedro Ruiz',    'pedro@email.com',  '3157891234', 'Barranquilla'),
 (5, 'Laura Gómez',   'laura@email.com',  '3003214567', 'Bogotá');
-- INSERT ... SELECT: insertar desde consulta
```

```
-- Copiar clientes con compras > 5M a tabla VIP
INSERT INTO clientes_vip (id_cliente, nombre, email, categoria, fecha_alta)
SELECT
   id_cliente,
   nombre,
   email,
   CASE
      WHEN total_compras > 10000000 THEN 'Platino'
      WHEN total_compras > 5000000  THEN 'Oro'
      ELSE 'Plata'
   END AS categoria,
   GETDATE() AS fecha_alta
FROM clientes
WHERE total_compras > 5000000
AND   estado = 'activo';
```

Al trabajar con el comando INSERT es fundamental tener en cuenta las restricciones de integridad definidas en la tabla: las columnas definidas como NOT NULL deben recibir un valor (o tener un valor por defecto definido), las claves primarias no pueden duplicarse, y las claves foráneas deben hacer referencia a valores que existan en la tabla referenciada. Cuando alguna de estas restricciones se viola, el SMBD lanzará

un error y revertirá la inserción, protegiendo así la integridad de los datos. Es responsabilidad del desarrollador manejar correctamente estos errores en la capa de aplicación y proporcionar mensajes de error informativos al usuario.

B. Comando UPDATE — Modificación de registros

El comando UPDATE es el mecanismo del DML para modificar los valores de uno o más campos en los registros que cumplen una condición específica. Es uno de los comandos más poderosos y, al mismo tiempo, uno de los que requiere mayor cautela en su uso, porque si se ejecuta sin una cláusula WHERE correctamente definida, puede modificar todos los registros de la tabla de manera irreversible, con consecuencias potencialmente desastrosas en un entorno de producción.

La estructura básica del UPDATE es: UPDATE [tabla] SET [columna1 = valor1, columna2 = valor2, ...] WHERE [condición]. La cláusula SET puede modificar múltiples columnas en una sola sentencia, lo que es más eficiente que ejecutar múltiples UPDATE separados. La cláusula WHERE determina qué registros se van a actualizar; su ausencia o una condición demasiado amplia puede tener consecuencias no deseadas que van desde datos incorrectos hasta la pérdida total de información crítica del negocio.

El UPDATE también puede utilizar el valor actual de la columna en la expresión de actualización, lo que lo hace muy útil para operaciones como incrementar contadores, actualizar saldos, ajustar precios por un porcentaje o concatenar texto a un campo existente. Adicionalmente, la variante UPDATE con subconsulta o con JOIN permite actualizar registros de una tabla basándose en información de otra tabla, lo que es fundamental en operaciones de sincronización y actualización en lote.

Ejemplo:

```
-- UPDATE básico: actualizar un registro específico por clave primaria

UPDATE clientes

SET    telefono  = '3001111111',

       ciudad    = 'Bogotá D.C.',

       email     = 'ana.garcia.nueva@email.com'

WHERE  id_cliente = 1;

-- UPDATE con expresión: incrementar precio 10% a productos de Electrónica

UPDATE productos

SET    precio     = precio * 1.10,

       fecha_mod  = GETDATE(),

       usuario_mod = 'sistema'

WHERE  id_categoria IN (

    SELECT id_categoria FROM categorias WHERE nombre = 'Electrónica'

);

-- UPDATE con CASE: actualizar categoría según total de compras

UPDATE clientes

SET    categoria = CASE

    WHEN total_compras >= 10000000 THEN 'Platino'
```

```
    WHEN total_compras >= 5000000  THEN 'Oro'
    WHEN total_compras >= 1000000  THEN 'Plata'
    ELSE 'Estándar'
END
WHERE  estado = 'activo';
-- UPDATE con JOIN (SQL Server): actualizar usando datos de otra tabla
UPDATE p
SET    p.stock_minimo = c.cantidad_promedio * 0.20
FROM   productos p
JOIN   (SELECT id_producto, AVG(cantidad) AS cantidad_promedio
        FROM   detalle_pedidos
        GROUP BY id_producto) c ON p.id_producto = c.id_producto;
```

Una práctica de seguridad ampliamente recomendada es envolver los UPDATE críticos en una transacción explícita (BEGIN TRANSACTION) y verificar los resultados antes de confirmar los cambios con COMMIT. De esta manera, si algo sale mal o si el número de registros afectados no es el esperado (verificable con la función @@ROWCOUNT en SQL Server o ROW_COUNT() en MySQL), se puede revertir la operación con ROLLBACK sin que los cambios queden registrados permanentemente. Esta práctica es especialmente importante en actualizaciones masivas o en entornos donde los datos son críticos para el negocio.

C. Comando DELETE — Eliminación de registros

El comando DELETE es la operación del DML para eliminar uno o más registros de una tabla que cumplen la condición especificada en la cláusula WHERE. Al igual que UPDATE, la omisión de la cláusula WHERE resulta en la eliminación de todos los registros de la tabla, razón por la cual muchas organizaciones implementan políticas que requieren aprobación explícita antes de ejecutar DELETE sin WHERE en entornos de producción.

Es importante comprender la diferencia entre DELETE, TRUNCATE TABLE y DROP TABLE, tres operaciones que pueden confundirse, pero tienen consecuencias muy diferentes. DELETE elimina los registros que cumplen la condición (o todos si no hay WHERE), activa los triggers definidos en la tabla, puede revertirse dentro de una transacción y registra cada eliminación en el log de transacciones. TRUNCATE TABLE elimina todos los registros de una tabla de manera mucho más eficiente (sin registrar cada fila en el log), pero no puede usarse con condiciones, no activa triggers y en la mayoría de los SMBD no puede revertirse fácilmente. DROP TABLE elimina la tabla completa junto con todos sus datos, índices y definiciones.

En muchos sistemas empresariales se implementa el patrón de eliminación lógica (soft delete) en lugar de la eliminación física: en lugar de ejecutar DELETE, se actualiza un campo booleano (como 'activo' o 'eliminado') que indica que el registro ya no debe aparecer en las consultas normales. Esta práctica preserva el historial de datos, facilita la auditoría, y permite recuperar registros eliminados por error. El costo es un mayor volumen de datos en la tabla y la necesidad de incluir siempre el filtro del campo de estado en todas las consultas.

Ejemplo:

```
-- DELETE básico: eliminar un registro específico

DELETE FROM clientes WHERE id_cliente = 10;

-- DELETE condicional: eliminar pedidos cancelados antes de 2023

DELETE FROM pedidos

WHERE  estado       = 'cancelado'

AND    fecha_pedido < '2023-01-01';

-- DELETE con subquery: eliminar clientes inactivos sin pedidos

DELETE FROM clientes

WHERE  estado = 'inactivo'

AND    id_cliente NOT IN (

   SELECT DISTINCT id_cliente

   FROM   pedidos

   WHERE  id_cliente IS NOT NULL

);

-- Patrón de eliminación lógica (soft delete) — recomendado en producción

UPDATE clientes

SET    eliminado    = 1,

       fecha_baja   = GETDATE(),
```

```
    usuario_baja = 'admin'
WHERE  id_cliente = 10;
-- DELETE con transacción explícita — buena práctica en producción
BEGIN TRANSACTION;
   DELETE FROM detalle_pedidos WHERE id_pedido = 500;
   DELETE FROM pedidos        WHERE id_pedido = 500;
   -- Verificar que se eliminaron los registros esperados
   IF @@ROWCOUNT = 1
      COMMIT TRANSACTION;
   ELSE
      ROLLBACK TRANSACTION;
```

Basado en lo anterior, la siguiente tabla resume los principales comandos de modificación del DML, incluyendo una variante de inserción y la instrucción TRUNCATE, con su sintaxis básica, el efecto que producen sobre la base de datos y las precauciones más importantes que debe tener el desarrollador al utilizarlos en entornos reales:

Tabla 1. Comandos de modificación DML: sintaxis, efecto y precauciones esenciales

Comando	Sintaxis básica	Efecto en la BD	Precaución principal	Reversible con ROLLBACK
INSERT.	INSERT INTO tabla (cols) VALUES (vals).	Agrega nuevos registros.	Respetar NOT NULL, claves	Sí, si está en transacción.

Comando	Sintaxis básica	Efecto en la BD	Precaución principal	Reversible con ROLLBACK
			primarias y foráneas.	
INSERT...SELECT.	INSERT INTO tabla SELECT ... FROM ...	Inserta resultado de consulta.	Verificar tipos y número de columnas coincidan.	Sí, si está en transacción.
UPDATE.	UPDATE tabla SET col=val WHERE cond.	Modifica registros existentes.	SIEMPRE usar WHERE; sin él actualiza toda la tabla.	Sí, si está en transacción.
DELETE.	DELETE FROM tabla WHERE cond.	Elimina registros de la tabla.	SIEMPRE usar WHERE; verificar integridad referencial.	Sí, si está en transacción.
TRUNCATE.	TRUNCATE TABLE tabla.	Elimina todos los registros.	No acepta WHERE; no activa `triggers`; muy rápido.	Limitado según el SMBD.

1.2. Búsqueda y recuperación de datos con el comando SELECT

El comando SELECT es, con diferencia, el más utilizado y el más versátil de todo el lenguaje SQL. Su función principal es recuperar datos de una o más tablas de la base de datos y presentarlos de la manera que el usuario o la aplicación requiera, ya sea como un conjunto de registros completos, como un subconjunto de columnas, como datos agrupados y resumidos, o como el resultado de combinar múltiples tablas relacionadas. A diferencia de INSERT, UPDATE y DELETE, SELECT no modifica el estado de la base de datos: es una operación de solo lectura que puede ejecutarse con total seguridad en

cualquier momento, sin riesgo de alterar los datos almacenados, razón por la cual es la operación más frecuente en cualquier aplicación que trabaje con bases de datos.

La potencia del SELECT radica en su capacidad de combinar múltiples cláusulas en una sola sentencia para filtrar, ordenar, agrupar, calcular y presentar datos de maneras muy sofisticadas. Una sentencia SELECT bien construida puede reemplazar decenas de líneas de código de aplicación, realizando en el motor de base de datos cálculos que serían mucho más lentos e ineficientes si se hicieran en la capa de aplicación después de recuperar los datos crudos. Esta es una de las razones fundamentales por las que aprender a escribir sentencias SELECT eficientes y correctas es una de las competencias más valiosas que un desarrollador de software puede realizar a lo largo de su carrera profesional.

La estructura general de una sentencia SELECT sigue un orden fijo de cláusulas que el motor de base de datos procesa en un orden lógico interno diferente al orden de escritura. Comprender este orden de procesamiento es fundamental para escribir consultas correctas y evitar errores comunes como usar alias definidos en SELECT dentro del WHERE (lo que no está permitido porque SELECT se procesa después de WHERE), o incluir funciones de agregación en WHERE en lugar de HAVING. El orden de escritura es: SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY. El orden de procesamiento lógico es: FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

Ejemplo:

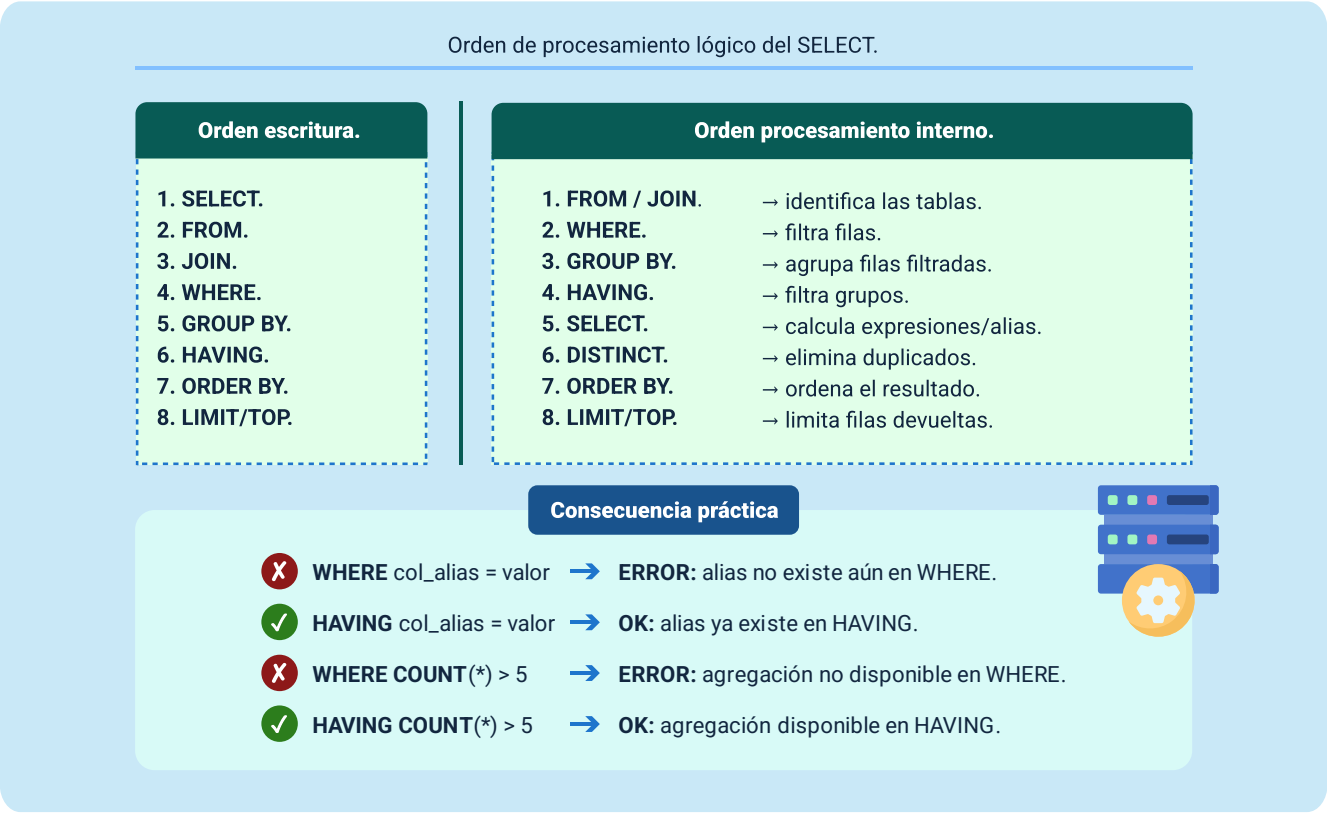
```
-- Estructura completa de una sentencia SELECT (orden de escritura)
SELECT   [ALL | DISTINCT] columnas, expresiones, funciones
FROM     tabla_principal
```

```
[LEFT | RIGHT | INNER | FULL] JOIN tabla_secundaria ON condición_join
[WHERE   condición_filtro_filas]
[GROUP BY columna(s) de agrupación]
[HAVING  condición_filtro_grupos]
[ORDER BY columna(s) [ASC | DESC]]
[TOP n | LIMIT n | FETCH FIRST n ROWS ONLY];
-- Ejemplo real: reporte de ventas por ciudad y categoría
SELECT
   c.ciudad                    AS ciudad,
   cat.nombre                  AS categoria,
   COUNT(DISTINCT p.id_pedido)      AS num_pedidos,
   COUNT(DISTINCT p.id_cliente)     AS clientes_distintos,
   SUM(dp.cantidad * dp.precio_unidad) AS ventas_brutas,
   AVG(dp.cantidad * dp.precio_unidad) AS ticket_promedio
FROM   clientes         c
JOIN   pedidos          p   ON c.id_cliente   = p.id_cliente
JOIN   detalle_pedidos  dp  ON p.id_pedido    = dp.id_pedido
JOIN   productos        pr  ON dp.id_producto = pr.id_producto
JOIN   categorias       cat ON pr.id_categoria= cat.id_categoria
```

```
WHERE  p.estado = 'completado'
AND    YEAR(p.fecha_pedido) = 2024
GROUP BY c.ciudad, cat.nombre
HAVING SUM(dp.cantidad * dp.precio_unidad) > 500000
ORDER BY ventas_brutas DESC;
```

De igual manera, para entender el tema de ordenamiento, se presenta la siguiente imagen. Es importante comprender dicho orden para evitar errores de lógica en consultas complejas:

Figura 2. Orden de procesamiento lógico interno del comando SELECT



Para finalizar, la siguiente tabla presenta ejemplos progresivos del comando SELECT, desde la forma más simple hasta consultas con múltiples cláusulas combinadas, ilustrando cómo cada cláusula adicional enriquece y refina el resultado de la consulta:

Tabla 2. Ejemplos progresivos del comando SELECT — de básico a avanzado

Nivel	Sentencia SQL (simplificada)	Resultado obtenido	Nuevas cláusulas
1 - Todos.	SELECT * FROM clientes.	Todos los campos y registros de clientes.	FROM.
2 - Proyección.	SELECT nombre, email FROM clientes.	Solo las columnas nombre y email.	SELECT cols.
3 - Filtrado.	SELECT nombre FROM clientes WHERE ciudad='Bogotá'.	Solo clientes de Bogotá.	WHERE.
4 - Ordenado.	SELECT nombre FROM clientes ORDER BY nombre ASC.	Clientes en orden alfabético.	ORDER BY.
5 - Sin duplicados.	SELECT DISTINCT ciudad FROM clientes.	Lista única de ciudades sin repetir.	DISTINCT.
6 - Con alias.	SELECT nombre AS cliente, email AS correo FROM clientes.	Columnas con nombres alternativos.	AS.
7 - Limitado.	SELECT TOP 10 nombre FROM clientes ORDER BY total_compras DESC.	Los 10 mejores clientes.	TOP/LIMIT.
8 - Agrupado.	SELECT ciudad, COUNT(*) AS total FROM clientes GROUP BY ciudad.	Cantidad de clientes por ciudad.	GROUP BY.
9 - Filtro grupo.	...GROUP BY ciudad HAVING COUNT(*) > 5.	Solo ciudades con más de 5 clientes.	HAVING.

Nivel	Sentencia SQL (simplificada)	Resultado obtenido	Nuevas cláusulas
10 - Completo.	SELECT+FROM+JOIN+WHERE+GROUP BY+HAVING+ORDER BY.	Reporte multidimensional completo.	Todo combinado.

2. Sintaxis y cláusulas del comando SELECT

El comando SELECT es la herramienta más poderosa del lenguaje SQL y, por esa misma razón, la que requiere un estudio más detallado y sistemático. Su capacidad para combinar múltiples cláusulas en una sola sentencia permite resolver problemas de consulta de datos de alta complejidad con elegancia y eficiencia. Para dominar el SELECT es fundamental comprender no solo la sintaxis de cada cláusula individualmente, sino también cómo interactúan entre sí, en qué orden lógico se procesan y qué restricciones impone cada cláusula sobre las demás.

Las cláusulas del SELECT no son solo herramientas técnicas, son el vocabulario con el que el desarrollador formula preguntas precisas sobre los datos del negocio: ¿cuáles son los diez productos más vendidos del último trimestre?, ¿qué ciudades tienen un promedio de ventas superior a un millón de pesos?, ¿cuántos clientes nuevos se registraron por mes durante el año pasado? Cada una de estas preguntas de negocio puede traducirse directamente en una sentencia SELECT con las cláusulas apropiadas. La capacidad de hacer esta traducción—de la pregunta de negocio a la sentencia SQL—de manera natural y eficiente es la marca del desarrollador con dominio real del lenguaje.

En este tema se estudian en profundidad todas las cláusulas del comando SELECT que el profesional utilizará con mayor frecuencia en su vida laboral.

2.1. Cláusulas de selección y filtrado de datos

Las cláusulas de selección y filtrado permiten refinar los resultados de una consulta SQL, controlando la unicidad de los datos, asignando nombres más

descriptivos y definiendo condiciones para recuperar únicamente la información requerida. Su uso adecuado mejora tanto la precisión de las consultas como la legibilidad del código.

A. ALL y DISTINCT — Control de unicidad en el resultado

Las palabras clave ALL y DISTINCT son modificadores que se colocan inmediatamente después de la palabra SELECT y controlan si el resultado de la consulta incluye o no filas duplicadas. Ambas pueden representar lo siguiente:

- **ALL**: es el comportamiento predeterminado del comando SELECT y rara vez se escribe de forma explícita, ya que el motor SQL devuelve todas las filas que cumplen la condición establecida, incluyendo registros repetidos o duplicados cuando estos existen en los datos consultados.
- **DISTINCT**: elimina las filas duplicadas del resultado y presenta únicamente valores únicos. Esto resulta especialmente útil en consultas de análisis, generación de reportes y procesos donde se requiere identificar información sin repeticiones.

Es importante entender que DISTINCT opera sobre la combinación completa de todas las columnas especificadas en el SELECT, no sobre una sola columna en particular. Si se especifican tres columnas en el SELECT, DISTINCT eliminará solo las filas donde las tres columnas tienen simultáneamente los mismos valores. Esto significa que el resultado de un SELECT DISTINCT puede tener más filas de las que el programador novato anticipa, especialmente cuando se incluyen columnas con alta cardinalidad (muchos valores distintos).

DISTINCT también puede utilizarse dentro de las funciones de agregación, como COUNT(DISTINCT columna), para contar solo los valores únicos de una columna específica. Esta variante es muy útil para responder preguntas como: ¿cuántos clientes distintos realizaron pedidos este mes? o ¿cuántas categorías de productos diferentes se vendieron en la región?

Ejemplo:

```
-- DISTINCT sobre una columna: lista de ciudades únicas

SELECT DISTINCT ciudad

FROM   clientes

ORDER BY ciudad;

-- DISTINCT sobre múltiples columnas: combinaciones únicas

SELECT DISTINCT ciudad, departamento

FROM   clientes

ORDER BY departamento, ciudad;

-- Solo elimina filas donde ciudad Y departamento son iguales

-- COUNT con DISTINCT: cuántos clientes distintos compraron este mes

SELECT

   COUNT(*)                AS total_pedidos,

   COUNT(DISTINCT id_cliente)  AS clientes_distintos,

   COUNT(DISTINCT id_producto) AS productos_distintos
```

```
FROM detalle_pedidos dp

JOIN pedidos p ON dp.id_pedido = p.id_pedido

WHERE MONTH(p.fecha_pedido) = MONTH(GETDATE())

AND   YEAR(p.fecha_pedido)  = YEAR(GETDATE());
```

B. Alias con AS — Renombrar columnas y tablas

La cláusula AS permite asignar nombres alternativos, llamados alias, tanto a columnas como a tablas dentro de una consulta. Los alias son temporales —solo existen durante la ejecución de la consulta— pero tienen un impacto significativo en la legibilidad del código SQL y en la usabilidad de los resultados para la aplicación o el usuario que los consume.

A continuación, se detallan los alias:

- **Alias de columna**: son especialmente útiles cuando el nombre original de la columna es críptico (como nombres heredados de sistemas legados), cuando se utiliza una expresión calculada que no tiene nombre natural (como precio * cantidad), o cuando se desea presentar los datos con un nombre más descriptivo y orientado al usuario final. En la mayoría de los SMBD, si el alias contiene espacios o caracteres especiales, debe encerrarse entre comillas dobles o corchetes según el dialecto SQL utilizado.
- **Alias de tabla**: son fundamentales cuando se trabaja con múltiples tablas en la misma consulta, especialmente en JOINs, porque permiten evitar la ambigüedad cuando dos tablas tienen columnas con el mismo nombre.

También acortan significativamente la escritura de las sentencias cuando los nombres de las tablas son largos, reduciendo el riesgo de errores tipográficos. Es una buena práctica usar alias de tabla cortos y nemotécnicos (como 'c' para clientes, 'p' para pedidos, 'pr' para productos) que faciliten la lectura de la consulta.

Ejemplo:

```
-- Alias de columna: nombres descriptivos para el usuario
SELECT
    id_cliente                    AS 'Código Cliente',
    nombre                        AS 'Nombre Completo',
    email                         AS 'Correo Electrónico',
    total_compras                 AS 'Total Histórico (COP)',
    total_compras * 0.19          AS 'IVA Estimado',
    total_compras + (total_compras * 0.19)  AS 'Total con IVA'
FROM clientes
WHERE estado = 'activo';
-- Alias de tabla: consulta multi-tabla con nombres cortos
SELECT
    c.nombre       AS cliente,
```

```
    c.ciudad       AS ciudad,
    p.fecha_pedido  AS fecha,
    p.total        AS valor_pedido,
    pr.nombre      AS producto,
    dp.cantidad    AS unidades
FROM   clientes       c
JOIN   pedidos        p   ON c.id_cliente   = p.id_cliente
JOIN   detalle_pedidos dp  ON p.id_pedido    = dp.id_pedido
JOIN   productos      pr  ON dp.id_producto = pr.id_producto
WHERE  p.estado = 'completado'
ORDER BY p.fecha_pedido DESC;
```

C. Cláusula WHERE y sus operadores de filtrado

La cláusula WHERE es el mecanismo de filtrado de filas del SELECT. Permite especificar condiciones que deben cumplir las filas para ser incluidas en el resultado de la consulta. WHERE acepta cualquier expresión lógica que evalúe a verdadero (TRUE), falso (FALSE) o desconocido (NULL) para cada fila de la tabla, e incluye solo las filas donde la condición evalúa a TRUE.

Las condiciones de WHERE pueden combinarse con los operadores lógicos AND (ambas condiciones deben ser verdaderas), OR (al menos una condición debe ser verdadera) y NOT (niega la condición). Cuando se utilizan varios operadores lógicos en la misma condición, el orden de precedencia es: primero NOT, luego AND, luego OR.

Para garantizar el orden de evaluación deseado y hacer el código más legible, es recomendable usar paréntesis explícitamente, aunque no sean estrictamente necesarios.

Además de los operadores de comparación estándar (=, <>, >, <, >=, <=), SQL ofrece tres operadores especiales de filtrado que amplían significativamente las posibilidades de búsqueda:

- **IN**: verifica si un valor pertenece a una lista de valores predefinidos o al resultado de una subconsulta.
- **BETWEEN**: verifica si un valor se encuentra dentro de un rango inclusivo.
- **LIKE**: permite búsquedas por patrones de texto utilizando los metacaracteres % (cero o más caracteres) y _ (exactamente un carácter).

Ejemplo:

```
-- Operadores de comparación básicos

SELECT nombre, precio FROM productos

WHERE  precio >= 50000 AND precio <= 200000;  -- equivalente a BETWEEN

-- Operador IN: pertenencia a una lista de valores

SELECT nombre, ciudad, telefono FROM clientes

WHERE  ciudad IN ('Bogotá', 'Medellín', 'Cali', 'Barranquilla', 'Cartagena');

-- Operador NOT IN: exclusión de valores

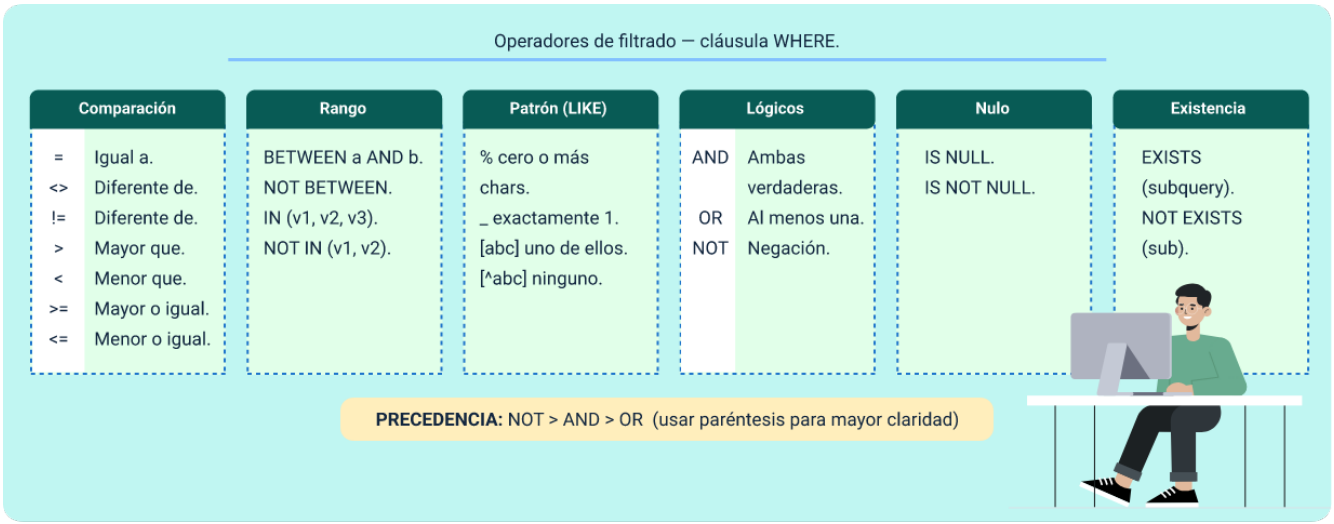
SELECT nombre FROM clientes

WHERE  estado NOT IN ('inactivo', 'bloqueado', 'eliminado');
```

```
-- Operador BETWEEN: rango de fechas
SELECT * FROM pedidos
WHERE  fecha_pedido BETWEEN '2024-01-01' AND '2024-03-31'  -- Q1 2024
AND    total BETWEEN 100000 AND 5000000;
-- Operador LIKE: búsqueda por patrones de texto
SELECT * FROM clientes WHERE nombre  LIKE 'Ana%';      -- Inicia con Ana
SELECT * FROM clientes WHERE email   LIKE '%@gmail%';  -- Contiene @gmail
SELECT * FROM clientes WHERE telefono LIKE '310_______'; -- 10 dígitos inicia 310
SELECT * FROM clientes WHERE nombre  LIKE '%García%'; -- Contiene García
-- Combinación de múltiples condiciones con AND, OR y NOT
SELECT nombre, ciudad, total_compras, estado
FROM   clientes
WHERE  (ciudad = 'Bogotá' OR ciudad = 'Medellín' OR ciudad = 'Cali')
AND    total_compras BETWEEN 500000 AND 10000000
AND    estado NOT IN ('inactivo', 'bloqueado')
AND    fecha_registro >= '2022-01-01'
ORDER BY total_compras DESC;
```

Es importante analizar la siguiente imagen representativa:

Figura 3. Operadores de filtrado en la cláusula WHERE — referencia completa



2.2. Agrupación, ordenamiento y condiciones sobre grupos

Las cláusulas de agrupación, filtrado de grupos y ordenamiento permiten transformar consultas simples en análisis de datos más completos y estructurados. Mediante GROUP BY, HAVING y ORDER BY es posible resumir información, aplicar condiciones sobre resultados agregados y presentar los datos en un orden significativo, facilitando la generación de reportes y el análisis de información en SQL.

A. GROUP BY — Agrupación de datos

La cláusula GROUP BY es el mecanismo fundamental del SQL para realizar análisis agregados, ya que permite agrupar las filas del resultado que tienen los mismos valores en las columnas especificadas y aplicar funciones de agregación (COUNT, SUM, AVG, MAX, MIN) sobre cada grupo, obteniendo un valor resumen por grupo en lugar de un valor por fila individual. Esta capacidad transforma el SELECT de una herramienta de

búsqueda de registros individuales en una herramienta de análisis de datos de alto nivel.

Una regla fundamental que el desarrollador debe memorizar es que cuando se utiliza GROUP BY, todas las columnas que aparecen en el SELECT pero que no están dentro de una función de agregación deben estar listadas obligatoriamente en la cláusula GROUP BY. El SMBD no puede devolver un valor individual para una columna no agregada cuando hay múltiples filas en el grupo, porque tendría que elegir arbitrariamente uno de los valores disponibles. Esta regla, aunque puede parecer restrictiva al principio, garantiza la coherencia lógica del resultado.

GROUP BY puede utilizarse con múltiples columnas, lo que produce grupos definidos por la combinación única de valores de todas las columnas especificadas. Así, una instrucción como GROUP BY ciudad, departamento genera un grupo por cada combinación única de ciudad y departamento presente en los datos. Esta capacidad permite realizar análisis multidimensionales orientados a responder preguntas como: ¿cuáles son las ventas totales por categoría de producto y por región del país?

Ejemplo:

```
-- GROUP BY simple: total de clientes y ventas por ciudad
SELECT
   ciudad,
   COUNT(*)          AS total_clientes,
   SUM(total_compras) AS ventas_acumuladas,
   AVG(total_compras) AS venta_promedio,
```

```
    MAX(total_compras) AS mayor_compra,
    MIN(total_compras) AS menor_compra
FROM   clientes
WHERE  estado = 'activo'
GROUP BY ciudad
ORDER BY ventas_acumuladas DESC;
-- GROUP BY múltiple: ventas por año y mes
SELECT
    YEAR(fecha_pedido)  AS año,
    MONTH(fecha_pedido) AS mes,
    COUNT(*)          AS num_pedidos,
    SUM(total)        AS ventas_mes
FROM   pedidos
WHERE  estado = 'completado'
GROUP BY YEAR(fecha_pedido), MONTH(fecha_pedido)
ORDER BY año, mes;
```

B. HAVING — Filtrado de grupos

La cláusula HAVING actúa como el filtro de grupos: mientras WHERE filtra filas individuales antes del agrupamiento, HAVING filtra los grupos generados por GROUP BY

después del agrupamiento y el cálculo de las funciones de agregación. Esta distinción es fundamental y uno de los errores más comunes en SQL es intentar usar funciones de agregación en la cláusula WHERE, lo que genera un error de sintaxis porque en el momento de evaluación del WHERE aún no se han calculado los grupos.

La regla práctica para recordar la diferencia es:

- **WHERE**: filtra filas (registros individuales, antes de agrupar).
- **HAVING**: filtra grupos (resultados de la agregación, después de agrupar).

Cualquier condición que involucre una función de agregación (COUNT, SUM, AVG, MAX, MIN) debe ir en HAVING, no en WHERE. Las condiciones sobre columnas individuales (sin funciones de agregación) pueden ir en WHERE o en HAVING, pero es más eficiente ponerlas en WHERE porque así se reduce el número de filas antes de hacer el agrupamiento.

Ejemplo:

```
-- HAVING: filtrar grupos con condición de agregación
SELECT
   ciudad,
   COUNT(*)         AS total_clientes,
   SUM(total_compras) AS ventas_totales,
   AVG(total_compras) AS venta_promedio
FROM   clientes
WHERE  estado = 'activo'         -- filtra FILAS (antes de agrupar)
```

```
GROUP BY ciudad
HAVING COUNT(*) >= 5            -- filtra GRUPOS (después de agrupar)
AND    SUM(total_compras) > 5000000  -- solo ciudades con ventas > 5M
AND    AVG(total_compras) > 500000   -- y promedio > 500K
ORDER BY ventas_totales DESC;
-- Diferencia WHERE vs HAVING — error común vs. forma correcta
-- ❌ INCORRECTO: función de agregación en WHERE
-- SELECT ciudad, COUNT(*) FROM clientes
-- WHERE COUNT(*) > 5 GROUP BY ciudad;  -- ERROR
-- ✓ CORRECTO: función de agregación en HAVING
SELECT ciudad, COUNT(*) AS total
FROM   clientes
GROUP BY ciudad
HAVING COUNT(*) > 5;  -- OK
```

C. ORDER BY — Ordenamiento de resultados

La cláusula ORDER BY define el orden en que se presentan las filas del resultado de la consulta. Sin ORDER BY, el SMBD no garantiza ningún orden específico en el resultado, incluso si la consulta ejecutada varias veces devuelve los datos en el mismo orden en la práctica: este comportamiento no está garantizado por el estándar SQL y

puede cambiar con actualizaciones del motor, cambios en los índices o variaciones en el volumen de datos.

ORDER BY puede especificar múltiples columnas de ordenamiento, cada una con dirección ascendente (ASC, que es el valor por defecto) o descendente (DESC). Cuando se especifican múltiples columnas, el ordenamiento se aplica de izquierda a derecha: primero se ordena por la primera columna, luego las filas con el mismo valor en la primera columna se ordenan por la segunda columna, y así sucesivamente. Esta capacidad de ordenamiento multinivel es fundamental para producir listados ordenados y reportes profesionales.

Ejemplo:

```
-- ORDER BY simple: ascendente por defecto
SELECT nombre, ciudad FROM clientes ORDER BY nombre;  -- A-Z
-- ORDER BY múltiple: ciudad A-Z, luego compras de mayor a menor
SELECT nombre, ciudad, total_compras
FROM   clientes
ORDER BY ciudad ASC, total_compras DESC;
-- ORDER BY con expresión calculada
SELECT nombre, total_compras,
     total_compras * 0.19 AS iva
FROM   clientes
ORDER BY iva DESC;  -- Ordenar por columna calculada
```

- ORDER BY con NULLS al final (útil para datos incompletos)

```
SELECT nombre, fecha_ultimo_pedido
FROM   clientes
ORDER BY fecha_ultimo_pedido DESC NULLS LAST;  -- PostgreSQL
```

Para finalizar, la siguiente tabla sintetiza todas las cláusulas del SELECT con su función, el momento de procesamiento en el que actúan y los errores más comunes que cometen los desarrolladores novatos con cada una:

Tabla 3. Cláusulas del comando SELECT: función, orden de procesamiento y errores comunes

Cláusula	Función principal	Orden procesamiento	Error más frecuente
SELECT.	Define columnas, expresiones y alias del resultado.	5°	Usar alias de SELECT dentro del WHERE (no disponible aún).
FROM / JOIN.	Especifica tablas fuente y sus relaciones.	1°	Omitir alias cuando hay columnas con el mismo nombre en varias tablas.
WHERE.	Filtra filas individuales antes del agrupamiento.	2°	Usar funciones de agregación en WHERE (deben ir en HAVING).
GROUP BY.	Agrupa filas con valores iguales en las columnas.	3°	Omitir en GROUP BY columnas del SELECT que no están en agregaciones.
HAVING.	Filtra los grupos resultantes del GROUP BY.	4°	Poner condiciones de filas individuales en HAVING (son más eficientes en WHERE).

Cláusula	Función principal	Orden procesamiento	Error más frecuente
ORDER BY.	Ordena el conjunto resultado final.	7°	Asumir orden natural sin ORDER BY (no garantizado por el estándar SQL).
DISTINCT.	Elimina filas completamente duplicadas del resultado.	6°	Creer que DISTINCT afecta solo una columna (afecta la combinación completa).
TOP / LIMIT.	Limita el número de filas devueltas.	8°	Usar sin ORDER BY (las filas devueltas son no determinísticas).

3. Funciones para la manipulación y análisis de datos

Las funciones SQL son rutinas predefinidas que el motor de base de datos pone a disposición del programador para realizar operaciones sobre los datos directamente en el servidor, sin necesidad de recuperarlos primero a la aplicación para procesarlos. Esta capacidad de procesar datos en el servidor, donde residen, es una de las ventajas más significativas del enfoque relacional: permite realizar transformaciones complejas sobre millones de registros con una eficiencia que sería imposible de lograr procesando los datos en la capa de aplicación, especialmente cuando se trabaja con grandes volúmenes de información.

El principio que guía el uso de funciones SQL en el servidor es el de reducir el tráfico de red entre la aplicación y la base de datos. En lugar de recuperar millones de registros a la aplicación para filtrar, transformar y calcular en memoria, es mucho más eficiente expresar esas operaciones en SQL y dejar que el motor de base de datos las ejecute donde están los datos, devolviendo solo el resultado final. Este principio, conocido como compute-close-to-data, es especialmente relevante en sistemas distribuidos y aplicaciones de big data, pero aplica igualmente en sistemas transaccionales convencionales.

SQL ofrece una amplia variedad de funciones según el tipo de dato que procesan y el tipo de resultado que producen. En esta temática se estudian las dos categorías más utilizadas en el desarrollo de aplicaciones: las funciones de cadena (o de texto), que operan sobre valores de tipo VARCHAR o CHAR y permiten manipular, transformar y extraer partes de cadenas de texto; y las funciones de agregación, que operan sobre conjuntos de filas y calculan un único valor resumen para cada grupo. Además de estas dos categorías, SQL también ofrece funciones de fecha y tiempo, funciones

matemáticas, funciones de conversión de tipos y funciones condicionales como CASE y COALESCE, que son igualmente importantes en la práctica profesional.

3.1. Funciones de cadena

Las funciones de cadena permiten manipular y transformar valores de texto almacenados en la base de datos. Son especialmente útiles para cuatro propósitos principales: normalizar datos ingresados por usuarios (que pueden llegar en mayúsculas, minúsculas o con formatos inconsistentes), extraer partes específicas de cadenas (como los primeros N caracteres de un código de producto o el dominio de una dirección de email), reemplazar caracteres o subcadenas no deseadas (como limpiar caracteres especiales o estandarizar separadores), y construir representaciones de texto dinámicas en los resultados de las consultas (como concatenar el nombre y apellido de una persona en un único campo de presentación).

Es importante tener en cuenta que la sintaxis de algunas funciones de cadena varía entre los diferentes dialectos de SQL (T-SQL de SQL Server, MySQL, PostgreSQL, Oracle PL/SQL). Aunque las funciones fundamentales existen en todos los SMBD, sus nombres y parámetros pueden diferir. Por ejemplo, la función de extracción de subcadena se llama SUBSTRING en SQL Server y MySQL, pero SUBSTR en Oracle y está disponible también como SUBSTRING en PostgreSQL. Al desarrollar código SQL que deba ser portable entre diferentes SMBD, es recomendable utilizar las funciones del estándar SQL siempre que sea posible.

A. LOWER y UPPER — Conversión de mayúsculas y minúsculas

Las funciones LOWER y UPPER son las más simples y utilizadas de las funciones de cadena:

- **LOWER**: convierte todos los caracteres alfabéticos de una cadena de texto a letras minúsculas, manteniendo sin cambios números, espacios y caracteres especiales. Esta función es útil para estandarizar datos y realizar comparaciones sin sensibilidad a mayúsculas o minúsculas.
- **UPPER**: transforma todos los caracteres alfabéticos de una cadena a letras mayúsculas, conservando intactos los demás símbolos y números. Se utiliza frecuentemente para normalizar información, generar formatos consistentes y facilitar búsquedas o validaciones de texto.

Son ampliamente utilizadas para normalizar datos antes de realizar comparaciones o búsquedas, evitando que diferencias en el uso de mayúsculas y minúsculas generen resultados incorrectos en los filtros o en las operaciones de JOIN. Así, si una columna de email puede contener valores como 'ANA@EMAIL.COM', 'ana@email.com' o 'Ana@Email.Com', una comparación directa con '=' fallaría en dos de los tres casos; utilizando LOWER en ambos lados de la comparación se garantiza el resultado correcto.

Ejemplo:

```
-- LOWER: normalización para búsqueda insensible a mayúsculas

SELECT id_cliente, nombre, email

FROM   clientes
```

```
WHERE  LOWER(email) = LOWER('ANA.GARCIA@EMAIL.COM');

-- UPPER: presentar código en mayúsculas siempre

SELECT UPPER(codigo_producto) AS codigo, nombre, precio

FROM   productos

ORDER BY codigo;

-- Combinación para formato Título (primera letra mayúscula)

-- SQL Server: no tiene función nativa, se construye con concatenación

SELECT

   nombre,

   UPPER(LEFT(nombre,1)) + LOWER(SUBSTRING(nombre,2,LEN(nombre))) AS
nombre_titulo

FROM clientes;
```

B. REPLACE — Reemplazo de subcadenas

La función REPLACE reemplaza todas las ocurrencias de una subcadena buscada dentro de una cadena por otra subcadena de reemplazo. Es una herramienta fundamental para la limpieza y normalización de datos que provienen de fuentes externas o de entradas de usuario, donde frecuentemente se encuentran caracteres no deseados, separadores inconsistentes o formatos que difieren del estándar interno del sistema. REPLACE es sensible a mayúsculas y minúsculas en la mayoría de los SMBD,

aunque este comportamiento puede depender de la collation (reglas de comparación de texto) definida para la columna o la base de datos.

Ejemplo:

```
-- Eliminar guiones de números de teléfono
SELECT nombre,
      REPLACE(telefono, '-', '') AS telefono_limpio
FROM   clientes;
-- '300-123-4567' → '3001234567'
-- REPLACE anidado: múltiples caracteres en una sola expresión
SELECT
   REPLACE(
      REPLACE(
         REPLACE(descripcion, ',', ';'),
      '.', ''),
   '  ', ' ') AS descripcion_limpia
FROM productos;
-- Reemplazar en UPDATE: estandarizar formato de emails
UPDATE clientes
SET    email = LOWER(REPLACE(email, ' ', ''))  -- minúsculas, sin espacios
```

WHERE email LIKE '% %' OR email <> LOWER(email);

C. SUBSTRING, LEFT y RIGHT — Extracción de subcadenas

Las funciones SUBSTRING (o SUBSTR), LEFT y RIGHT permiten extraer una porción específica de una cadena de texto, así:

- **SUBSTRING**: extrae una subcadena a partir de una posición de inicio y con una longitud determinada.
- **LEFT**: extrae los primeros N caracteres desde el extremo izquierdo de la cadena.
- **RIGHT**: extrae los últimos N caracteres desde el extremo derecho.

Estas funciones son esenciales para trabajar con campos que almacenan información codificada en formato de cadena, como códigos de producto con estructura definida, referencias contables o identificadores compuestos.

Ejemplo:

```
-- SUBSTRING: extraer componentes de un código estructurado
-- Código formato: 'CAT-PRD-2024-001' (categoría-producto-año-secuencia)
SELECT
   codigo_producto,
   SUBSTRING(codigo_producto, 1, 3)   AS categoria_cod,
   SUBSTRING(codigo_producto, 5, 3)   AS tipo_producto,
   SUBSTRING(codigo_producto, 9, 4)   AS año_creacion,
```

```
    RIGHT(codigo_producto, 3)        AS secuencia
FROM productos;
-- LEFT: extraer dominio de email (todo hasta el @)
SELECT
    email,
    LEFT(email, CHARINDEX('@', email) - 1) AS usuario,
    SUBSTRING(email, CHARINDEX('@', email) + 1, LEN(email)) AS dominio
FROM clientes;
-- RIGHT + LEN: validar longitud de campos
SELECT nombre, LEN(nombre) AS longitud,
      CASE WHEN LEN(nombre) > 50 THEN 'LARGO' ELSE 'OK' END AS estado
FROM   clientes
ORDER BY longitud DESC;
```

D. STR y funciones de conversión

La función STR convierte un valor numérico a su representación de texto con un formato específico de longitud total y número de decimales. Es útil para presentar valores numéricos con formato fijo en reportes o cuando se necesita concatenar números con texto. En la mayoría de los SMBD modernos, la función CAST o CONVERT ofrece mayor flexibilidad y control sobre la conversión de tipos de datos. La función

CONCAT (o el operador + en SQL Server) permite unir múltiples cadenas o valores convertidos en una sola cadena de texto.

Ejemplo:

```
-- STR: número a texto con formato fijo
SELECT nombre, STR(precio, 12, 2) AS precio_formateado
FROM   productos;  -- '  50000.00', '1500000.00'
-- CONCAT: concatenar campos para presentación
SELECT CONCAT(nombre, ' (', ciudad, ')') AS cliente_ciudad
FROM   clientes;  -- 'Ana García (Bogotá)'
-- CAST: conversión de tipos — número a texto
SELECT 'Pedido #' + CAST(id_pedido AS VARCHAR(10)) +
      ' del ' + CONVERT(VARCHAR, fecha_pedido, 103) AS referencia
FROM   pedidos;
-- 'Pedido #1234 del 15/03/2024'
```

La siguiente tabla presenta un resumen completo de referencia rápida de todas las funciones de cadena estudiadas con su sintaxis y un ejemplo representativo de uso en contextos de desarrollo empresarial:

Tabla 4. Funciones de cadena SQL: referencia rápida de sintaxis y ejemplos

Función	Sintaxis	Descripción	Ejemplo — Entrada → Salida
LOWER.	LOWER(cadena).	Convierte a minúsculas.	LOWER('COLOMBIA') → 'colombia'.
UPPER.	UPPER(cadena).	Convierte a mayúsculas.	UPPER('bogotá') → 'BOGOTÁ'.
REPLACE.	REPLACE(cad, buscar, nuevo).	Reemplaza todas las ocurrencias.	REPLACE('300-123','–','') → '300123'.
SUBSTRING.	SUBSTRING(cad, inicio, longitud).	Extrae subcadena por posición.	SUBSTRING('CF04-SQL',1,4) → 'CF04'.
LEFT.	LEFT(cadena, n).	Primeros n caracteres.	LEFT('Bogotá D.C.',6) → 'Bogotá'.
RIGHT.	RIGHT(cadena, n).	Últimos n caracteres.	RIGHT('Colombia',5) → 'ombia'.
LEN / LENGTH.	LEN(cadena).	Número de caracteres.	LEN('SENA') → 4.
STR.	STR(numero, long, dec).	Número a cadena con formato.	STR(3.14159, 8, 2) → ' 3.14'.
LTRIM / RTRIM.	LTRIM(cad) / RTRIM(cad).	Elimina espacios extremos.	LTRIM(' Hola') → 'Hola'.
TRIM.	TRIM(cadena).	Elimina espacios en ambos lados.	TRIM(' Hola ') → 'Hola'.
CONCAT.	CONCAT(cad1, cad2, ...).	Concatena múltiples cadenas.	CONCAT('Ana', ' ', 'García') → 'Ana García'.
CHARINDEX.	CHARINDEX(buscar, cadena).	Posición de la primera ocurrencia.	CHARINDEX('@','ana@mail') → 4.

3.2. Funciones de agregación

Las funciones de agregación son el núcleo del SQL analítico, ya que operan sobre un conjunto de filas —que puede ser toda la tabla o un grupo definido por GROUP BY— y devuelven un único valor resumen para ese conjunto. Son el mecanismo principal del SQL para realizar análisis estadísticos y de negocio directamente en la base de datos, sin necesidad de exportar los datos a herramientas externas. Calcular el total de ventas por región, el promedio de calificaciones por estudiante, el precio máximo por categoría de producto, o el número de pedidos por cliente en el último mes: todos estos análisis se realizan de manera natural y eficiente con las funciones de agregación.

Un aspecto fundamental para entender las funciones de agregación es su comportamiento frente a los valores NULL. Con la excepción de COUNT(*) que cuenta todas las filas incluyendo aquellas con valores NULL en cualquier columna, todas las demás funciones de agregación ignoran los valores NULL en sus cálculos. Esto significa que AVG calcula el promedio de los valores no nulos (lo que puede producir resultados diferentes del promedio sobre todos los registros si se consideran los NULL como cero), SUM suma solo los valores no nulos, y MAX y MIN ignoran los NULL al buscar el valor extremo. Comprender este comportamiento es crítico para garantizar la corrección de los análisis.

A. COUNT—Conteo de registros y valores

La función COUNT tiene tres variantes principales con comportamientos diferentes:

- **COUNT(*)**: cuenta todas las filas del grupo, incluyendo aquellas con valores NULL en cualquier columna.

- **COUNT(columna)**: cuenta solo las filas donde el valor de esa columna específica no es NULL.
- **COUNT(DISTINCT columna)**: cuenta el número de valores únicos y no nulos de esa columna.

La elección entre estas variantes debe hacerse conscientemente según el análisis que se desea realizar.

Ejemplo:

```
-- Las tres variantes de COUNT y sus diferencias
SELECT
   COUNT(*)                AS total_filas,      -- incluye NULLs
   COUNT(id_cliente)       AS filas_con_cliente, -- excluye NULLs
   COUNT(DISTINCT id_cliente)  AS clientes_distintos -- valores únicos
FROM pedidos
WHERE YEAR(fecha_pedido) = 2024;
-- COUNT por grupos: pedidos por estado
SELECT
   estado,
   COUNT(*) AS cantidad,
   COUNT(*) * 100.0 / SUM(COUNT(*)) OVER() AS porcentaje
FROM pedidos
```

GROUP BY estado

ORDER BY cantidad DESC;

B. SUM, AVG, MAX y MIN — Análisis numérico

Las funciones SUM, AVG, MAX y MIN son las herramientas fundamentales para el análisis cuantitativo de datos. SUM calcula la suma total de los valores de una columna para el grupo, ignorando los NULL. AVG calcula el promedio aritmético de los valores no nulos. MAX y MIN devuelven el valor máximo y mínimo respectivamente, y funcionan no solo con datos numéricos sino también con fechas (devuelven la fecha más reciente o más antigua) y con cadenas de texto (devuelven el valor mayor o menor según el orden alfabético o de collation definido).

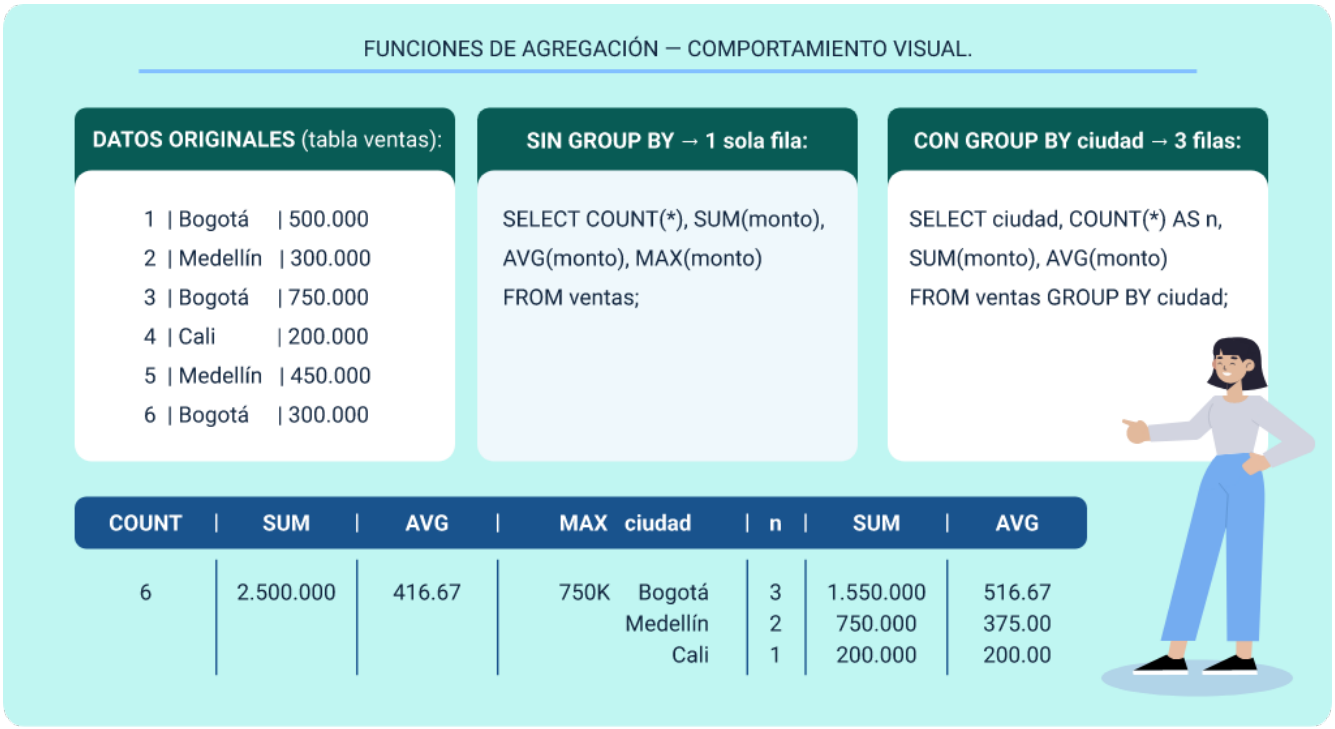
Ejemplo:

```
-- Análisis completo de ventas por categoría
SELECT
    cat.nombre                    AS categoria,
    COUNT(DISTINCT dp.id_pedido)  AS num_pedidos,
    SUM(dp.cantidad)              AS unidades_vendidas,
    SUM(dp.cantidad * dp.precio_unidad) AS ingresos_brutos,
    AVG(dp.precio_unidad)         AS precio_promedio,
    MIN(dp.precio_unidad)         AS precio_minimo,
    MAX(dp.precio_unidad)         AS precio_maximo,
```

```
    MAX(dp.precio_unidad) - MIN(dp.precio_unidad) AS rango_precio
FROM   detalle_pedidos dp
JOIN   productos      p   ON dp.id_producto  = p.id_producto
JOIN   categorias     cat ON p.id_categoria  = cat.id_categoria
GROUP BY cat.nombre
HAVING SUM(dp.cantidad * dp.precio_unidad) > 1000000
ORDER BY ingresos_brutos DESC;
```

Para finalizar, se presenta la siguiente imagen comparativa:

Figura 4. Comportamiento de las funciones de agregación — sin vs. con GROUP BY



De igual manera, se presenta la siguiente tabla que resume las funciones de agregación con su descripción completa, comportamiento frente a NULL y ejemplos de uso en análisis empresariales reales del contexto colombiano:

Tabla 5. Funciones de agregación SQL: referencia completa de comportamiento

Función	Descripción	Comportamiento con NULL	Ejemplo de uso analítico
COUNT(*).	Cuenta todas las filas del grupo.	Incluye filas con NULL en cualquier columna.	Total de pedidos registrados en el período.
COUNT(col).	Cuenta filas donde col no es NULL.	Excluye las filas donde col es NULL.	Pedidos que tienen dirección de envío registrada.
COUNT(DISTINCT col).	Cuenta valores únicos no nulos de col.	Excluye NULLs y duplicados.	Clientes distintos que compraron este mes.
SUM(col).	Suma los valores no nulos de col.	Ignora NULLs completamente.	Total de ventas brutas por región del país.
AVG(col).	Promedio de los valores no nulos.	Ignora NULLs (denominador = filas no nulas).	Precio promedio de venta por categoría.
MAX(col).	Valor máximo (número, fecha o texto).	Ignora NULLs al comparar.	Fecha del pedido más reciente por cliente.
MIN(col).	Valor mínimo (número, fecha o texto).	Ignora NULLs al comparar.	Precio mínimo de producto activo por proveedor.

4. Consultas avanzadas en SQL

Una vez que el desarrollador domina los fundamentos del SELECT y las funciones SQL, el siguiente nivel de competencia consiste en aprender a construir sentencias avanzadas que resuelvan problemas de mayor complejidad en el acceso a datos. Esto incluye situaciones en las que el resultado de una consulta depende de los resultados de otra (subconsultas), o en las que es necesario combinar y relacionar información proveniente de múltiples tablas (JOINs). Estas dos capacidades —subconsultas y JOINs—constituyen el núcleo del SQL avanzado y son las que más claramente diferencian a un desarrollador con dominio profundo de bases de datos de uno con un conocimiento básico del lenguaje.

Las subconsultas añaden un nivel de expresividad que permite al SQL responder preguntas del tipo: ¿qué clientes compraron todos los productos de la categoría X? o ¿qué productos tienen un precio superior al precio promedio de su categoría? —preguntas que requieren que el resultado de una consulta sea utilizado como parte de otra consulta. Los JOINs, por su parte, implementan la operación más característica del modelo relacional, ya que recomponen en una sola vista cohesiva la información que el principio de normalización ha distribuido en múltiples tablas para eliminar redundancias y garantizar la integridad de los datos.

Juntas, estas técnicas permiten responder a prácticamente cualquier pregunta que se pueda formular sobre los datos almacenados en una base de datos relacional. La clave para utilizarlas efectivamente está en comprender con precisión qué tipo de resultado produce cada operador, en qué circunstancias es más apropiado usar subconsultas versus JOINs, y cómo el SMBD optimiza internamente estas operaciones para garantizar el mejor rendimiento posible.

4.1. Subconsultas y tipos de subconsultas

Una subconsulta (también llamada subquery o consulta anidada) es una sentencia SELECT que se integra dentro de otra instrucción SQL, como SELECT, INSERT, UPDATE o DELETE. La instrucción externa se conoce como consulta principal, mientras que la interna recibe el nombre de subconsulta. En términos de ejecución, el motor de base de datos suele evaluar primero la parte interna y utilizar su resultado para completar el procesamiento de la consulta principal, aunque los optimizadores modernos pueden reorganizar estas operaciones si identifican un plan más eficiente.

Estas estructuras pueden clasificarse de distintas maneras:

- **Según su ubicación dentro de la sentencia SQL**

Pueden aparecer en la cláusula WHERE, donde se utilizan para filtrar resultados (las más habituales); en la cláusula FROM, funcionando como tablas derivadas que permiten construir conjuntos de datos intermedios; o en la lista del SELECT, donde actúan como subconsultas escalares que aportan valores calculados al resultado final.

- **De acuerdo con el tipo de resultado que producen**

Se distinguen subconsultas escalares, que devuelven un único valor; subconsultas de fila, que retornan una fila completa con múltiples columnas; y subconsultas de tabla, que producen conjuntos de múltiples filas y columnas que pueden ser utilizados por la consulta principal.

- **Según su dependencia de la consulta exterior**

Pueden ser no correlacionadas, cuando se evalúan una sola vez y su resultado se reutiliza, lo que suele ofrecer un mejor rendimiento; o correlacionadas, cuando

dependen de valores de la consulta exterior y deben reevaluarse para cada fila, lo que incrementa su costo computacional, pero permite expresar condiciones más complejas.

La elección entre utilizar subconsultas o JOINs para resolver un mismo problema suele implicar consideraciones de estilo y rendimiento. En la mayoría de los motores modernos de bases de datos (como SQL Server, PostgreSQL o MySQL 8.0+), los optimizadores son capaces de transformar automáticamente ciertas subconsultas en JOINs cuando ello conduce a un plan de ejecución más eficiente. No obstante, existen escenarios en los que las primeras resultan más expresivas y legibles —especialmente en combinaciones con EXISTS—y otros en los que los JOINs ofrecen claras ventajas tanto en claridad como en desempeño.

A. Subconsultas con IN y NOT IN

El operador IN con subconsulta es la forma más intuitiva de usar subconsultas, porque verifica si un valor de la consulta exterior existe en el conjunto de valores devueltos por la subconsulta interior. Es conceptualmente equivalente a una condición OR extendida sobre todos los valores del conjunto resultado. NOT IN verifica que el valor no pertenezca al conjunto. Una precaución importante: si la subconsulta de NOT IN puede devolver valores NULL, toda la condición puede producir resultados inesperados porque NULL NOT IN (conjunto con NULL) evalúa a NULL (no a TRUE ni FALSE) y por tanto, ninguna fila pasaría el filtro. Por esta razón, cuando se usa NOT IN con subconsultas, es importante asegurarse de que la subconsulta filtre los valores NULL explícitamente.

Ejemplo:

```
-- IN: clientes que realizaron al menos un pedido en 2024
SELECT nombre, ciudad, email
FROM   clientes
WHERE  id_cliente IN (
   SELECT DISTINCT id_cliente
   FROM   pedidos
   WHERE  YEAR(fecha_pedido) = 2024
   AND    estado = 'completado'
);
-- NOT IN: clientes que NUNCA han realizado un pedido
-- (atención: filtrar NULLs en la subconsulta)
SELECT nombre, email, fecha_registro
FROM   clientes
WHERE  id_cliente NOT IN (
   SELECT DISTINCT id_cliente
   FROM   pedidos
   WHERE  id_cliente IS NOT NULL  -- crítico para NOT IN
)
```

```
ORDER BY fecha_registro DESC;

-- IN con subconsulta compleja: productos comprados por clientes VIP
SELECT DISTINCT pr.nombre, pr.precio
FROM   productos pr
WHERE  pr.id_producto IN (
   SELECT dp.id_producto
   FROM   detalle_pedidos dp
   JOIN   pedidos p ON dp.id_pedido = p.id_pedido
   WHERE  p.id_cliente IN (
      SELECT id_cliente FROM clientes WHERE categoria = 'Platino'
   )
);
```

B. Subconsultas con ANY, SOME y ALL

Los operadores ANY (o su sinónimo SOME) y ALL permiten comparar un valor con todos los elementos del conjunto devuelto por una subconsulta. ANY devuelve TRUE si la comparación es verdadera para al menos uno de los valores del conjunto (es decir, si la condición se cumple con alguno de los valores). ALL devuelve TRUE solo si la comparación es verdadera para todos los valores del conjunto (la condición se cumple con todos). Estos operadores combinados con los operadores de comparación (=, <>, >, <, >=, <=) proporcionan una gran flexibilidad para expresar condiciones que de otra manera requerirían funciones de agregación como MIN o MAX.

Ejemplo:

```
-- ANY: productos más caros que algún producto de Electrónica
SELECT nombre, precio FROM productos
WHERE  precio > ANY (
   SELECT precio FROM productos
   WHERE  id_categoria = (SELECT id_categoria FROM categorias WHERE nombre='Electrónica')
);
-- Equivalente a: precio > MIN(precios de Electrónica)
-- ALL: productos más caros que TODOS los de Electrónica
SELECT nombre, precio FROM productos
WHERE  precio > ALL (
   SELECT precio FROM productos
   WHERE  id_categoria = (SELECT id_categoria FROM categorias WHERE nombre='Electrónica')
);
-- Equivalente a: precio > MAX(precios de Electrónica)


-- = ANY: equivalente exacto a IN
```

```
SELECT nombre FROM clientes

WHERE  ciudad = ANY ('Bogotá','Medellín','Cali');  -- igual que IN
```

C. Subconsultas con EXISTS y NOT EXISTS

El operador EXISTS verifica si la subconsulta devuelve al menos una fila, sin importar los valores específicos de esas filas. La subconsulta de EXISTS generalmente se escribe como SELECT 1 (o SELECT *) para indicar que no se necesita recuperar ningún valor en particular, solo verificar si existe al menos un registro que cumpla la condición. EXISTS tiende a ser más eficiente que IN cuando la subconsulta devuelve muchas filas, porque el SMBD puede detener la búsqueda en cuanto encuentra la primera fila que cumple la condición, en lugar de generar todo el conjunto de valores.

NOT EXISTS verifica que la subconsulta no devuelva ninguna fila, siendo la alternativa más robusta a NOT IN cuando hay riesgo de valores NULL en el conjunto de comparación, porque EXISTS y NOT EXISTS no tienen el problema de los NULL que afecta a IN y NOT IN.

Ejemplo:

```
-- EXISTS: clientes que tienen al menos un pedido > 1.000.000

SELECT c.nombre, c.ciudad

FROM   clientes c

WHERE EXISTS (

   SELECT 1

   FROM   pedidos p
```

```
    WHERE  p.id_cliente = c.id_cliente  -- subconsulta correlacionada
    AND    p.total > 1000000
    AND    p.estado = 'completado'
);
-- NOT EXISTS: más robusto que NOT IN con posibles NULLs
SELECT c.nombre, c.email
FROM   clientes c
WHERE NOT EXISTS (
    SELECT 1
    FROM   pedidos p
    WHERE  p.id_cliente = c.id_cliente
);
-- Devuelve clientes sin ningún pedido (equivale al NOT IN pero sin problema de NULLs)
```

Partiendo de lo anterior, la siguiente tabla compara todos los operadores para subconsultas con su semántica precisa, cuándo usarlos preferiblemente y su equivalente lógico con funciones de agregación cuando aplica:

Tabla 6. Operadores para subconsultas: semántica, uso y equivalencias

Operador	Semántica	Cuándo preferirlo	Equivalente lógico / notas
IN (subquery).	TRUE si el valor está en el conjunto devuelto.	Lista dinámica de valores de otra tabla.	= v1 OR = v2 OR... ; eficiente si subconsulta devuelve pocos valores.
NOT IN (subquery).	TRUE si el valor NO está en el conjunto.	Exclusión dinámica; filtrar NULLs de subconsulta.	col<>v1 AND col<>v2... ; falla con NULLs si no se filtra explícitamente.
> ANY (subquery).	TRUE si mayor que al menos un valor.	Mayor que el mínimo del conjunto.	col > MIN(subquery); se puede reescribir con MIN.
> ALL (subquery).	TRUE si mayor que todos los valores.	Mayor que el máximo del conjunto.	col > MAX(subquery); se puede reescribir con MAX.
= ANY (subquery).	TRUE si igual a al menos un valor.	Sinónimo de IN; más explícito semánticamente.	Equivalente exacto a IN.
EXISTS (subquery).	TRUE si subconsulta devuelve al menos 1 fila.	Verificar existencia; muy eficiente con tablas grandes.	Detiene búsqueda al primer match; no tiene problema de NULLs.
NOT EXISTS (subquery).	TRUE si subconsulta devuelve 0 filas.	Preferible a NOT IN cuando hay riesgo de NULLs.	Alternativa robusta a NOT IN; no tiene problema de NULLs.

4.2. Consultas combinadas y tipos de JOIN

Los JOINs son la operación más característica del modelo relacional y el mecanismo mediante el cual el SQL implementa el concepto de relación entre tablas. La

normalización de bases de datos distribuye los datos en múltiples tablas para eliminar redundancias y garantizar la integridad: una tabla de pedidos no repite los datos del cliente en cada fila, sino que almacena solo el identificador (clave foránea) que hace referencia al registro correspondiente en la tabla de clientes. Los JOINs son el mecanismo que permite recomponer esa información distribuida en resultados cohesivos cuando se necesita consultarla de manera conjunta.

SQL define varios tipos de JOIN, cada uno con una semántica específica respecto a qué filas se incluyen en el resultado cuando no existe una fila correspondiente en una de las tablas involucradas. La elección del tipo de JOIN correcto determina si el resultado incluirá o no los registros que no tienen correspondencia, lo que a su vez determina si la consulta responde correctamente a la pregunta de negocio que se está formulando. Un JOIN incorrecto puede producir resultados que parecen razonables pero que omiten datos importantes o incluyen filas incorrectas.

El INNER JOIN es el tipo más común y produce solo las filas que tienen correspondencia en ambas tablas. Los OUTER JOINs (LEFT, RIGHT y FULL) incluyen las filas sin correspondencia en una o ambas tablas, completando con NULL los campos de la tabla que no tiene match. El CROSS JOIN produce el producto cartesiano (todas las combinaciones posibles) de los registros de ambas tablas. Cada tipo tiene sus casos de uso específicos en el contexto del desarrollo de aplicaciones empresariales.

Ejemplo:

```
-- INNER JOIN: solo pedidos que tienen cliente registrado

SELECT c.nombre, p.fecha_pedido, p.total

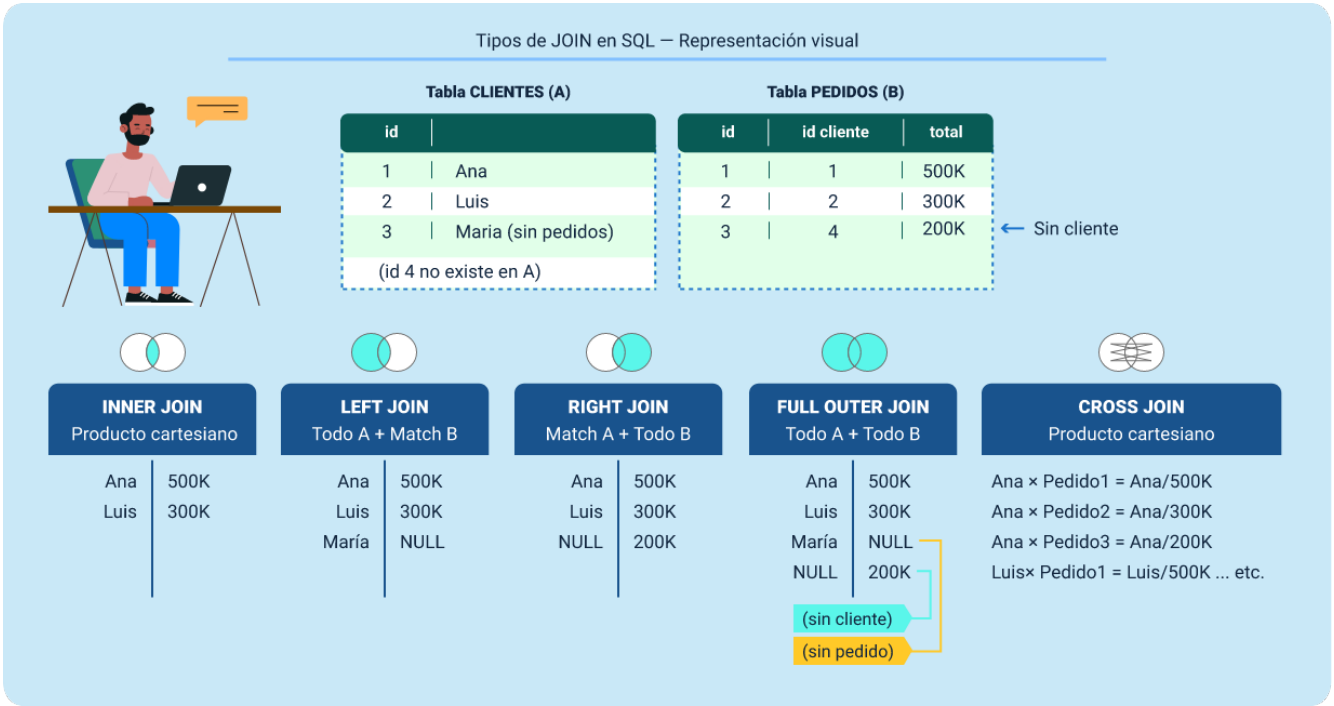
FROM   clientes c
```

```
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente
ORDER BY p.fecha_pedido DESC;
-- LEFT JOIN: TODOS los clientes, tengan o no pedidos
SELECT c.nombre, c.email,
    COUNT(p.id_pedido)    AS num_pedidos,
    COALESCE(SUM(p.total), 0) AS total_compras
FROM   clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nombre, c.email
ORDER BY total_compras DESC;
-- FULL OUTER JOIN: todos los clientes Y todos los pedidos
SELECT c.nombre, p.id_pedido, p.total
FROM   clientes c
FULL OUTER JOIN pedidos p ON c.id_cliente = p.id_cliente
WHERE  c.id_cliente IS NULL OR p.id_pedido IS NULL;
-- Solo muestra los registros SIN correspondencia en ambos lados
-- JOIN múltiple: cruzar cuatro tablas relacionadas
SELECT c.nombre, cat.nombre AS categoria,
    pr.nombre AS producto, dp.cantidad, dp.precio_unidad
```

```
FROM   clientes c
JOIN   pedidos p       ON c.id_cliente   = p.id_cliente
JOIN   detalle_pedidos dp ON p.id_pedido  = dp.id_pedido
JOIN   productos pr    ON dp.id_producto = pr.id_producto
JOIN   categorias cat  ON pr.id_categoria= cat.id_categoria
WHERE  YEAR(p.fecha_pedido) = 2024;
```

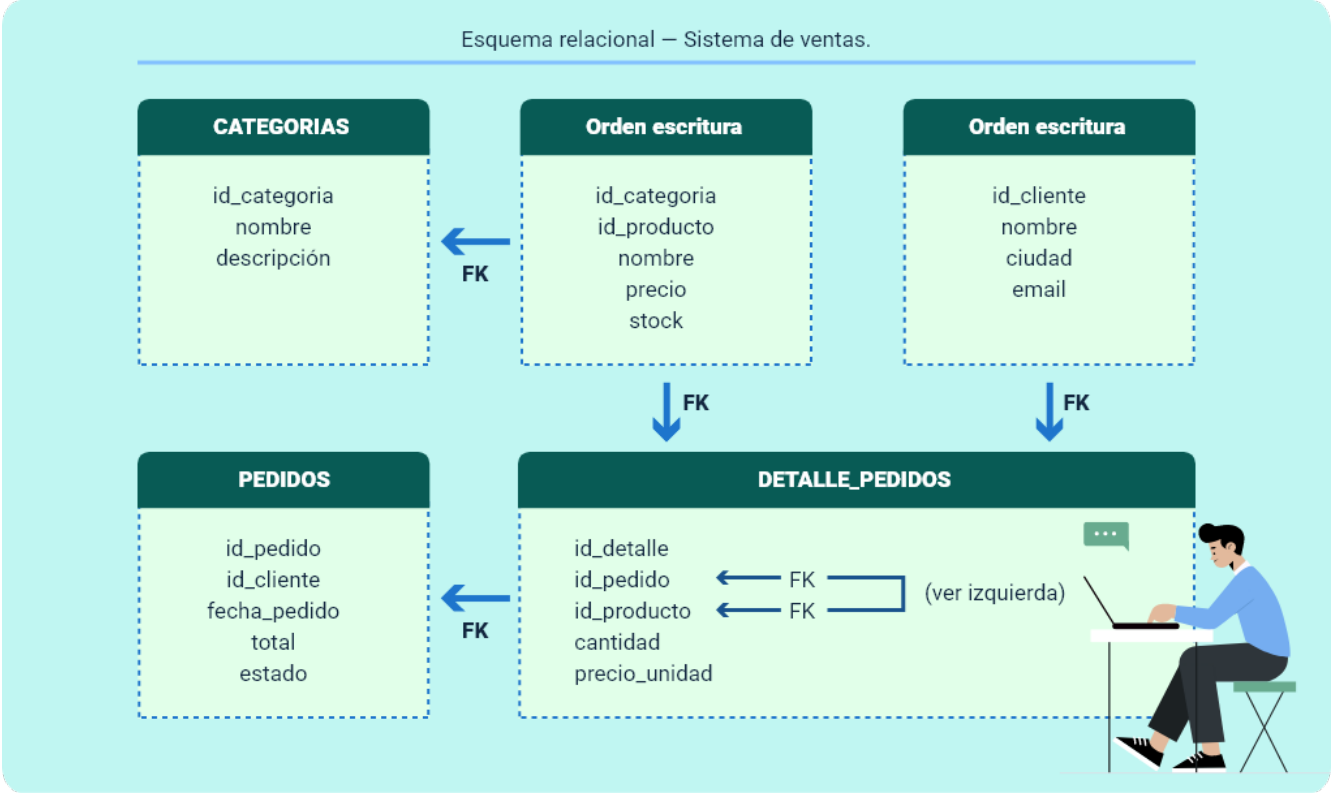
Es importante analizar la siguiente imagen que complementa lo explicado sobre los tipos de JOIN:

Figura 5. Tipos de JOIN en SQL — representación con datos de ejemplo



Igualmente, se presente la siguiente imagen que ejemplifica al respecto:

Figura 6. Esquema relacional de ejemplo — relaciones entre tablas y JOINs



De igual manera, se presentan las siguientes dos tablas que sintetizan los conceptos trabajados, permitiendo comparar tipos de estructuras SQL y sus usos más frecuentes en el contexto del SQL avanzado:

Tabla 7. Tipos de JOIN en SQL: descripción, condiciones de uso y ejemplos

Tipo de JOIN	Filas que incluye en el resultado	Caso típico de uso	Ejemplo empresarial
INNER JOIN.	Solo filas con correspondencia en AMBAS tablas — las más frecuentes.	La mayoría de consultas transaccionales.	Pedidos con sus datos de cliente (ambos deben existir).

Tipo de JOIN	Filas que incluye en el resultado	Caso típico de uso	Ejemplo empresarial
LEFT OUTER JOIN.	TODAS las filas de la tabla izquierda + correspondencias de la derecha (NULL si no hay).	Cuando necesitas todos los del lado izquierdo.	Todos los clientes, tengan o no pedidos — análisis de inactividad.
RIGHT OUTER JOIN.	Correspondencias de la izquierda + TODAS las de la tabla derecha (NULL si no hay).	Menos común; todos los del lado derecho.	Todos los productos, estén o no en pedidos actuales.
FULL OUTER JOIN.	TODAS las filas de AMBAS tablas, con NULL donde no hay correspondencia.	Auditoría o comparación de dos conjuntos completos.	Detectar registros huérfanos en ambas tablas simultáneamente.
CROSS JOIN.	Producto cartesiano — TODAS las combinaciones posibles de filas de ambas tablas.	Generar combinaciones exhaustivas entre dos catálogos.	Tabla de precios para todas las combinaciones talla-color-producto.

Tabla 8. Subconsultas vs JOINs: cuándo usar cada enfoque

Criterio	Subconsultas	JOINs
Legibilidad.	Alta cuando la lógica es jerárquica o condicional.	Alta cuando la relación entre tablas es directa.
Rendimiento.	Variable; puede ser lento con grandes conjuntos (especialmente correlacionadas).	Generalmente más eficiente; el optimizador los favorece.
Resultado.	Puede devolver el mismo conjunto de filas que el JOIN (depende del tipo).	Puede multiplicar filas si hay múltiples matches (cuidado con cardinalidad).

Criterio	Subconsultas	JOINs
EXISTS vs JOIN.	EXISTS es ideal para verificar existencia sin recuperar datos.	JOIN recupera datos; si solo se necesita verificar, EXISTS es mejor.
Casos donde subconsulta gana.	Lógica condicional compleja, NOT EXISTS, subconsultas escalares en SELECT.	Recuperar columnas de múltiples tablas simultáneamente.
Casos donde JOIN gana.	Recuperar muchas columnas de tablas relacionadas en un resultado plano.	Combinaciones donde se necesita el producto de varias tablas.

Como cierre de este tema, se presenta un video basado en un caso práctico que contextualiza el uso de los distintos tipos de JOIN en un escenario empresarial real. Este recurso permite aplicar los conceptos estudiados —relación entre tablas, elección del JOIN adecuado y análisis del resultado— reforzando la comprensión del impacto que tiene cada tipo de combinación en la respuesta a una necesidad de negocio concreta:

Video 1. JOINs en acción: resolviendo un caso real con SQL



Enlace de reproducción del video

Transcripción del video. JOINs en acción: resolviendo un caso real con SQL
Una empresa de ventas en línea necesita responder una pregunta clave para su operación: ¿qué clientes están comprando, cuáles no y qué pedidos presentan inconsistencias? Para ello, la información se encuentra distribuida en varias tablas relacionadas: clientes, pedidos, productos y detalle_pedidos. El primer paso es comprender que los datos no están duplicados, sino relacionados mediante claves primarias y foráneas. Aquí es donde los JOINs en SQL permiten reconstruir la información según la necesidad del análisis. Cuando el objetivo es listar solo los pedidos que tienen un cliente válido, se utiliza un INNER JOIN, ya que devuelve únicamente las coincidencias entre ambas tablas. Sin embargo, si se requiere identificar clientes sin pedidos, el enfoque cambia:

Transcripción del video. JOINs en acción: resolviendo un caso real con SQL
un LEFT JOIN permite incluir todos los clientes, incluso aquellos que no tienen registros asociados, mostrando valores NULL donde no hay correspondencia. Para tareas de auditoría, como detectar pedidos sin cliente o clientes sin pedidos, el FULL OUTER JOIN resulta fundamental, ya que muestra todas las filas de ambas tablas, revelando posibles inconsistencias de datos. En contraste, el CROSS JOIN se emplea solo en casos específicos, cuando se necesitan todas las combinaciones posibles entre dos conjuntos, como en la generación de catálogos. Este caso evidencia que elegir el tipo de JOIN correcto no es un detalle técnico, sino una decisión que impacta directamente la calidad de la información y la respuesta a las preguntas de negocio. Comprender esta lógica es clave para el uso efectivo del SQL avanzado.

5. Procedimientos almacenados en bases de datos

Un procedimiento almacenado (stored procedure) es un conjunto de sentencias SQL precompiladas y almacenadas directamente en el servidor de base de datos, que pueden ejecutarse como una unidad cohesiva mediante una simple llamada con nombre y parámetros opcionales. A diferencia de las consultas SQL que se envían dinámicamente desde la aplicación al servidor cada vez que se ejecutan —con el costo asociado de parsing, compilación y optimización en cada ejecución —los procedimientos almacenados residen permanentemente en el servidor y solo necesitan ser creados una vez para quedar disponibles de manera indefinida para todas las aplicaciones que tengan acceso al servidor de base de datos.

Los procedimientos almacenados representan uno de los mecanismos más poderosos para encapsular lógica de negocio en la capa de datos. Cuando una operación de negocio compleja requiere ejecutar múltiples sentencias SQL en secuencia, verificar condiciones intermedias, manejar errores y garantizar la atomicidad de toda la operación mediante transacciones, estos procedimientos ofrecen el entorno ideal: permiten expresar esa lógica de negocio de manera centralizada, con control total sobre el flujo de ejecución, el manejo de errores y la gestión de transacciones.

Desde una perspectiva arquitectónica, los procedimientos almacenados implementan el patrón de diseño conocido como “capa de acceso a datos” (Data Access Layer o DAL). En lugar de que cada aplicación que necesite acceder a los datos del negocio construya sus propias sentencias SQL, todas las aplicaciones llaman a procedimientos almacenados estandarizados. Esto garantiza la consistencia de las operaciones de negocio, facilita el mantenimiento (un cambio en la lógica del negocio se implementa en un solo lugar), mejora la seguridad (las aplicaciones solo tienen

permisos para ejecutar procedimientos, no para modificar tablas directamente) y reduce la superficie de ataque para inyecciones SQL.

En el ecosistema del desarrollo de software empresarial colombiano, los procedimientos almacenados son ampliamente utilizados en sistemas de gestión de inventarios, facturación electrónica (especialmente con los requisitos de la DIAN), nómina y contabilidad, sistemas bancarios y sistemas de salud, donde la complejidad de las reglas de negocio, los requisitos de auditoría y la necesidad de garantizar la integridad de las transacciones hacen de los stored procedures una herramienta invaluable que complementa perfectamente el código de aplicación.

5.1. Creación y definición de procedimientos almacenados

La creación de un procedimiento almacenado se realiza con el comando CREATE PROCEDURE (o CREATE PROC en la sintaxis abreviada de SQL Server). La sintaxis varía según el SMBD —T-SQL para SQL Server, PL/pgSQL para PostgreSQL, PL/SQL para Oracle, el dialecto propio de MySQL —pero la estructura conceptual es la misma en todos los casos: un nombre único para el procedimiento, una lista opcional de parámetros de entrada o salida con sus tipos de datos y el cuerpo del procedimiento delimitado por palabras clave específicas (AS BEGIN...END en SQL Server, BEGIN...END en MySQL después del DELIMITER).

El cuerpo de un procedimiento almacenado puede contener no solo sentencias DML (SELECT, INSERT, UPDATE, DELETE) sino también estructuras de control de flujo propias del lenguaje procedural del SMBD: condicionales (IF...ELSE, CASE), ciclos (WHILE), variables locales declaradas con DECLARE, cursores para procesar resultados

fila por fila, manejo de errores (TRY...CATCH en T-SQL, DECLARE HANDLER en MySQL), y llamadas a otros procedimientos almacenados o funciones. Esta capacidad de combinar SQL con lógica procedural es lo que convierte a los procedimientos almacenados en unidades de lógica de negocio completas y autosuficientes.

A. Estructura básica y variables locales

La estructura básica de un procedimiento almacenado comienza con el encabezado que define el nombre y los parámetros, seguido por el cuerpo delimitado por BEGIN y END. Dentro del cuerpo se pueden declarar variables locales usando DECLARE, que son visibles solo dentro del procedimiento y permiten almacenar resultados intermedios de cálculos o valores recuperados de la base de datos para usarlos en operaciones posteriores.

Ejemplo:

```
-- SQL Server / T-SQL: procedimiento básico con variables locales
CREATE PROCEDURE sp_ResumenCliente
    @id_cliente INT
AS
BEGIN
    SET NOCOUNT ON;  -- suprime el mensaje 'N rows affected'
    -- Variables locales para cálculos intermedios
    DECLARE @nombre       VARCHAR(100);
    DECLARE @total_pedidos INT;
```

```
DECLARE @ventas_totales DECIMAL(15,2);

DECLARE @categoria     VARCHAR(20);

-- Recuperar datos del cliente

SELECT @nombre = nombre FROM clientes WHERE id_cliente = @id_cliente;

-- Calcular métricas

SELECT @total_pedidos  = COUNT(*),

       @ventas_totales = SUM(total)

FROM   pedidos

WHERE  id_cliente = @id_cliente AND estado = 'completado';

-- Determinar categoría

SET @categoria = CASE

    WHEN @ventas_totales > 10000000 THEN 'Platino'

    WHEN @ventas_totales > 5000000  THEN 'Oro'

    WHEN @ventas_totales > 1000000  THEN 'Plata'

    ELSE 'Estándar'

END;

-- Retornar resultado

SELECT

    @nombre        AS cliente,
```

```
    @total_pedidos  AS num_pedidos,
    @ventas_totales AS ventas_totales,
    @categoria      AS categoria_sugerida;
END;
GO
```

B. Procedimientos con lógica de negocio compleja

Los procedimientos almacenados de uso real en entornos empresariales van mucho más allá de simples consultas parametrizadas. Encapsulan procesos de negocio completos que involucran múltiples operaciones secuenciales con validaciones intermedias, manejo de condiciones excepcionales y garantía de atomicidad mediante transacciones. Es así, como se detalla a continuación un procedimiento que implementa el proceso de registro de un pedido con verificación de `stock`, actualización de inventario y registro de la transacción, todo dentro de una transacción que garantiza que o todas las operaciones se completan exitosamente o ninguna se realiza.

Ejemplo:

```
CREATE PROCEDURE sp_RegistrarPedido
   @id_cliente   INT,
   @id_producto  INT,
   @cantidad     INT,
   @id_pedido    INT OUTPUT,   -- devuelve el ID generado
```

```
    @mensaje      VARCHAR(200) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @id_pedido = -1;
    SET @mensaje   = '';
    BEGIN TRANSACTION;
    BEGIN TRY
        DECLARE @stock       INT;
        DECLARE @precio      DECIMAL(12,2);
        DECLARE @cliente_ok  BIT;
        -- Validar que el cliente existe y está activo
        SELECT @cliente_ok = CASE WHEN estado='activo' THEN 1 ELSE 0 END
        FROM   clientes WHERE id_cliente = @id_cliente;
        IF @cliente_ok IS NULL BEGIN
            SET @mensaje = 'ERROR: Cliente no encontrado';
            ROLLBACK TRANSACTION; RETURN;
        END
        IF @cliente_ok = 0 BEGIN
```

```
    SET @mensaje = 'ERROR: Cliente inactivo o bloqueado';
    ROLLBACK TRANSACTION; RETURN;
END
-- Verificar stock disponible con bloqueo
SELECT @stock = stock, @precio = precio
FROM   productos WITH (UPDLOCK)
WHERE  id_producto = @id_producto AND activo = 1;
IF @stock IS NULL BEGIN
    SET @mensaje = 'ERROR: Producto no disponible';
    ROLLBACK TRANSACTION; RETURN;
END
IF @stock < @cantidad BEGIN
    SET @mensaje = 'ERROR: Stock insuficiente. Disponible: ' + CAST(@stock
   AS VARCHAR);
    ROLLBACK TRANSACTION; RETURN;
END
-- Registrar el pedido
INSERT INTO pedidos (id_cliente, fecha_pedido, total, estado)
VALUES (@id_cliente, GETDATE(), @precio * @cantidad, 'pendiente');
```

```
        SET @id_pedido = SCOPE_IDENTITY();

        -- Registrar el detalle

            INSERT INTO detalle_pedidos (id_pedido, id_producto, cantidad,
           precio_unidad)

        VALUES (@id_pedido, @id_producto, @cantidad, @precio);

        -- Descontar del inventario

        UPDATE productos

        SET    stock = stock - @cantidad

        WHERE  id_producto = @id_producto;

        SET @mensaje = 'Pedido #' + CAST(@id_pedido AS VARCHAR) + ' registrado
        OK';

        COMMIT TRANSACTION;

    END TRY

    BEGIN CATCH

        ROLLBACK TRANSACTION;

        SET @id_pedido = -99;

        SET @mensaje = 'ERROR inesperado: ' + ERROR_MESSAGE();

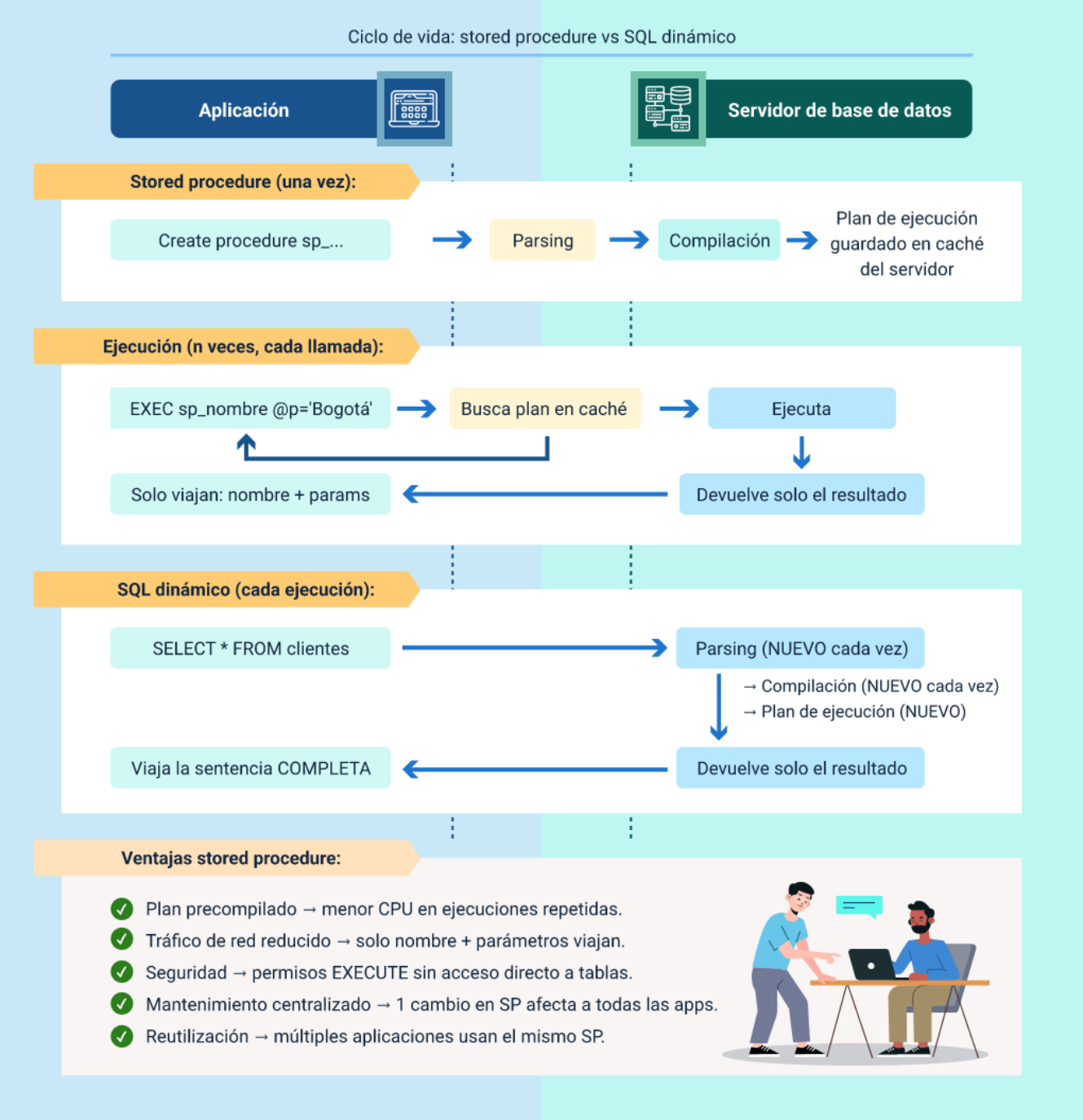
    END CATCH

END;
```

GO

Igualmente, se detalla la siguiente imagen explicativa del ciclo de vida de un procedimiento almacenado:

Figura 7. Ciclo de vida de los procedimientos almacenados vs SQL dinámico



5.2. Parámetros y ejecución de procedimientos

Los parámetros son el mecanismo de comunicación entre el código que realiza la llamada y el procedimiento. Permiten que este se comporte de manera diferente según los valores proporcionados, sin necesidad de crear versiones separadas para cada variación. Un diseño adecuado maximiza la reutilización mediante parámetros que cubren distintos escenarios de uso, incluyendo valores por defecto para los opcionales.

La gestión correcta de parámetros incluye también la validación de los valores recibidos: verificar que los parámetros obligatorios no sean NULL, que los valores numéricos estén en los rangos aceptables y que los identificadores proporcionados hagan referencia a registros que existan en las tablas correspondientes. Esta validación temprana (fail fast) evita que el procedimiento ejecute operaciones costosas solo para descubrir al final que los datos de entrada son inválidos.

Ejemplo:

```
-- Procedimiento con parámetros opcionales y valores por defecto
CREATE PROCEDURE sp_ReportePedidos
    @id_cliente  INT         = NULL,    -- NULL = todos los clientes
    @estado      VARCHAR(20)   = 'completado',
    @fecha_desde DATE         = NULL,    -- NULL = inicio del año actual
    @fecha_hasta DATE         = NULL,    -- NULL = hoy
    @top_n       INT          = 100      -- máximo de registros
AS
```

```
BEGIN
    SET NOCOUNT ON;
    -- Establecer valores por defecto para fechas
    SET @fecha_desde = ISNULL(@fecha_desde,
DATEFROMPARTS(YEAR(GETDATE()),1,1));
    SET @fecha_hasta = ISNULL(@fecha_hasta, CAST(GETDATE() AS DATE));
    SELECT TOP (@top_n)
        c.nombre     AS cliente,
        p.id_pedido,
        p.fecha_pedido,
        p.total,
        p.estado
    FROM   pedidos p
    JOIN   clientes c ON p.id_cliente = c.id_cliente
    WHERE  (@id_cliente IS NULL OR p.id_cliente = @id_cliente)
    AND    p.estado = @estado
    AND    p.fecha_pedido BETWEEN @fecha_desde AND @fecha_hasta
    ORDER BY p.fecha_pedido DESC;
        END;
```

```
GO

-- Diferentes formas de ejecutar el procedimiento

EXEC sp_ReportePedidos;                    -- todos los defaults

EXEC sp_ReportePedidos @id_cliente = 5;          -- cliente 5

EXEC sp_ReportePedidos @estado = 'pendiente';      -- pedidos pendientes

EXEC sp_ReportePedidos @fecha_desde = '2024-01-01',  -- rango específico

                @fecha_hasta = '2024-03-31',

                @top_n = 50;
```

A. Tipos de parámetros en procedimientos almacenados

Los procedimientos almacenados utilizan parámetros para recibir datos, devolver resultados adicionales y adaptar su comportamiento a distintos escenarios de uso. Aunque la sintaxis varía entre los principales SMBD, los tipos de parámetros responden a propósitos comunes que todo desarrollador debe conocer.

Los tipos principales de parámetros son:

- **Parámetros de entrada (IN)**: permiten recibir valores desde el código llamador y son el tipo por defecto en SQL Server, MySQL y PostgreSQL.
- **Parámetros de salida (OUT)**: devuelven valores calculados al finalizar la ejecución, complementando el conjunto de resultados principal.
- **Parámetros de entrada-salida (INOUT)**: reciben un valor inicial, lo modifican durante la ejecución y devuelven el valor actualizado al llamador.

- **Parámetros con valor por defecto**: facilitan llamadas más simples al usar un valor predefinido cuando el parámetro no es proporcionado explícitamente.

B. Gestión del ciclo de vida de los procedimientos almacenados

El trabajo con procedimientos almacenados implica una serie de operaciones que cubren todo su ciclo de vida, desde la creación hasta su mantenimiento y ejecución. Estas operaciones son fundamentales para administrar correctamente la lógica de negocio en la base de datos.

Las siguientes son las operaciones clave del ciclo de vida:

- **Creación**: definición inicial del procedimiento dentro del esquema de la base de datos, donde se establece su nombre, parámetros, lógica interna y propósito funcional.
- **Modificación**: actualización de la lógica interna del procedimiento para corregir errores, optimizar el rendimiento o incorporar nuevas reglas de negocio.
- **Eliminación**: borrado definitivo del procedimiento cuando deja de ser necesario, ha quedado obsoleto o es reemplazado por una nueva implementación.
- **Ejecución**: invocación del procedimiento desde aplicaciones, servicios o scripts SQL, proporcionando los parámetros requeridos para obtener el resultado esperado.
- **Listado**: consulta del catálogo del sistema para identificar los procedimientos existentes, verificar su disponibilidad y facilitar tareas de administración.

- **Visualización del código fuente**: inspección del código SQL que define el procedimiento, útil para análisis, depuración, auditoría o mantenimiento evolutivo.
- **Consulta de parámetros**: revisión de los parámetros definidos en el procedimiento, incluyendo sus tipos de datos, valores por defecto y direcciones (entrada, salida o entrada-salida).

C. Mejores prácticas para el desarrollo de procedimientos almacenados

El diseño profesional de procedimientos almacenados no solo busca que funcionen, sino que sean mantenibles, seguros y eficientes a largo plazo. Aplicar buenas prácticas reduce errores y facilita la evolución del sistema.

Como prácticas recomendadas se tienen:

- **Nomenclatura consistente y descriptiva**: definir un estándar de nombres claro (prefijos, verbos y entidades) permite identificar rápidamente el propósito del procedimiento, facilita la búsqueda en el catálogo y mejora la comprensión del código por parte de otros desarrolladores.
- **Validación temprana de parámetros**: comprobar valores nulos, rangos inválidos o formatos incorrectos al inicio del procedimiento evita ejecuciones innecesarias, reduce el consumo de recursos y permite detectar errores de forma temprana.
- **Uso de transacciones explícitas**: en operaciones que afectan múltiples tablas, el manejo explícito de transacciones garantiza la atomicidad y consistencia de los datos, asegurando que los cambios se confirmen o se reviertan de manera controlada ante fallos.

- **Manejo estructurado de errores**: implementar bloques de control de errores permite capturar excepciones, registrar información relevante y devolver mensajes claros al llamador, lo que mejora la confiabilidad y facilita el diagnóstico de problemas.
- **Documentación interna del procedimiento**: incluir comentarios sobre el propósito, los parámetros, las reglas de negocio y los cambios realizados facilita el mantenimiento a largo plazo y permite que cualquier miembro del equipo comprenda y modifique el procedimiento con menor riesgo.

6. Escenarios de almacenamiento y bases de datos en la nube

La computación en la nube ha transformado radicalmente la manera en que las organizaciones diseñan, despliegan, escalan y mantienen sus soluciones de almacenamiento de datos. Lo que hace apenas una década requería costosas inversiones en servidores físicos, licencias de software, personal especializado en administración de infraestructura y largas esperas para el aprovisionamiento de nuevos recursos, hoy puede realizarse en cuestión de minutos con unos pocos clics en una consola web, con un modelo de pago por uso que elimina las inversiones de capital iniciales y permite ajustar los recursos en tiempo real según la demanda.

Pero la nube no solo ha democratizado el acceso a la infraestructura de bases de datos, ha catalizado también una verdadera revolución en los paradigmas de almacenamiento de datos. El modelo relacional, dominante y prácticamente sin alternativas durante más de tres décadas, convive hoy con una amplia variedad de modelos de datos alternativos —conocidos colectivamente como bases de datos NoSQL (Not Only SQL) o bases de datos multimodelo— que están optimizados para casos de uso específicos donde las bases de datos relacionales presentan limitaciones inherentes de rendimiento, escalabilidad o flexibilidad. Esta diversificación del ecosistema de bases de datos es uno de los desarrollos más importantes de la última década en el campo de la ingeniería de datos.

Comprender este ecosistema diverso de opciones de almacenamiento, ya no es un lujo sino una necesidad. Los sistemas modernos frecuentemente combinan múltiples tipos de bases de datos en una sola arquitectura: una base de datos relacional para las transacciones del negocio, una base de datos de documentos para el catálogo de productos, una base de datos clave-valor para el caché de sesiones y una base de

datos de serie temporal para las métricas de monitoreo del sistema. Saber cuándo usar cada tipo y cómo integrar estas tecnologías es una competencia arquitectónica de alto valor en el mercado laboral actual.

6.1. Soluciones de almacenamiento en la nube y APIs multimodelo

Las tres principales plataformas de computación en la nube —Amazon Web Services (AWS), Microsoft Azure y Google Cloud Platform (GCP)— ofrecen hoy catálogos extensos de servicios de bases de datos gestionadas (Database as a Service, DBaaS), donde el proveedor se encarga de la infraestructura subyacente: instalación, configuración, parches de seguridad, respaldos automáticos, replicación para alta disponibilidad, monitoreo de rendimiento y escalado. El cliente solo se preocupa por diseñar el esquema de datos, escribir las consultas y gestionar los permisos de acceso.

Los servicios DBaaS eliminan algunas de las tareas más tediosas y especializadas de la administración de bases de datos tradicional, permitiendo que los equipos de desarrollo se concentren en el valor de negocio en lugar de en la gestión de la infraestructura. Sin embargo, también presentan ciertos trade-offs: menor control sobre la configuración detallada del motor, dependencia del proveedor de nube (vendor lock-in), y costos que pueden escalar rápidamente si no se dimensiona correctamente la capacidad contratada.

Las APIs multimodelo representan un paso adicional en la evolución de los servicios de bases de datos en la nube: son sistemas capaces de soportar múltiples modelos de datos (relacional, documental, clave-valor, grafo) sobre un único motor de almacenamiento subyacente, exponiendo APIs diferentes para cada paradigma. Azure

Cosmos DB es el ejemplo más conocido, ofreciendo cinco APIs compatibles: SQL (para consultas tipo JSON), MongoDB, Cassandra, Gremlin (grafos) y Table (clave-valor). Amazon Aurora ofrece compatibilidad con MySQL y PostgreSQL en el mismo servicio. Esta convergencia simplifica significativamente la arquitectura de datos de las organizaciones.

Ejemplo:

```
-- API SQL de Cosmos DB (documentos JSON con sintaxis similar a SQL)
SELECT c.nombre, c.ciudad, c.totalCompras
FROM   clientes c
WHERE  c.ciudad = 'Bogotá'
AND    c.totalCompras > 1000000
ORDER BY c.totalCompras DESC
OFFSET 0 LIMIT 10
-- API MongoDB (base de datos documental)
db.clientes.find(
   { ciudad: 'Bogotá', totalCompras: { $gt: 1000000 } },
   { nombre: 1, email: 1, totalCompras: 1 }
).sort({ totalCompras: -1 }).limit(10);
-- API Redis (clave-valor — caché de sesión de usuario)
SET    session:user:12345 '{"nombre":"Ana","rol":"admin"}' EX 3600
```

```
GET    session:user:12345

EXISTS session:user:12345

DEL    session:user:12345

-- Cypher (Neo4j/Cosmos DB Gremlin — base de datos de grafos)

MATCH (c:Cliente {nombre:'Ana'})-[:COMPRO]->(p:Producto)<-[:COMPRO]-
(otro:Cliente)

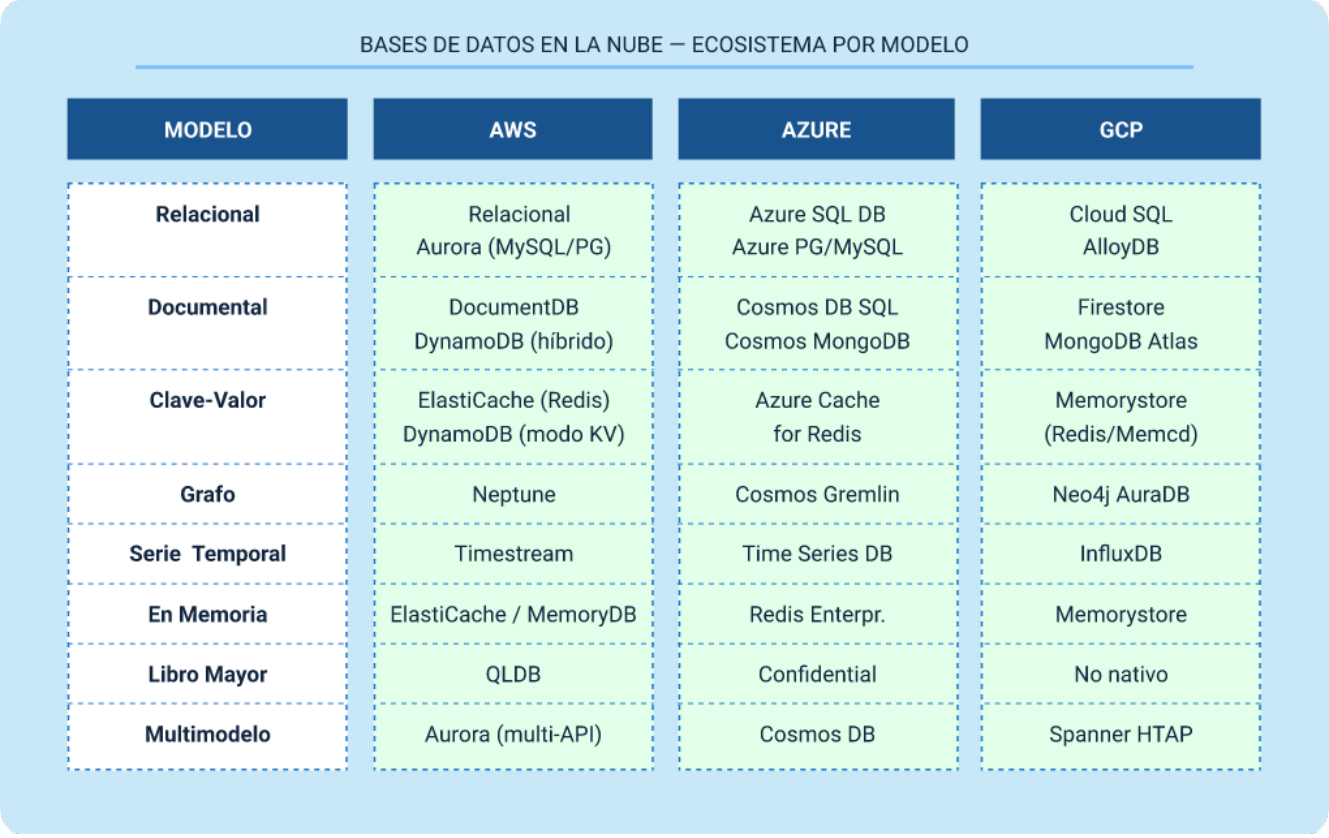
WHERE otro <> c

RETURN otro.nombre, COUNT(p) AS productos_en_comun

ORDER BY productos_en_comun DESC LIMIT 5;
```

Con base en lo anterior, la siguiente imagen presenta el ecosistema de bases de datos en la nube según el modelo de datos y el proveedor:

Figura 8. Ecosistema de bases de datos en la nube por modelo de datos y proveedor cloud



6.2. Tipos de bases de datos en entornos cloud

Los entornos cloud han ampliado significativamente el abanico de tecnologías de bases de datos disponibles para las organizaciones. Más allá del modelo relacional tradicional, hoy es posible elegir entre múltiples tipos de bases de datos optimizadas para diferentes patrones de acceso, volúmenes de información y requisitos de escalabilidad, rendimiento y consistencia, todo bajo el modelo de servicios gestionados.

En este contexto, los principales proveedores de nube ofrecen catálogos especializados que permiten seleccionar el modelo de datos más adecuado para cada

caso de uso, reduciendo la complejidad operativa y facilitando arquitecturas híbridas y multimodelo.

A continuación. se presentan los principales tipos de bases de datos disponibles en entornos cloud, describiendo sus características, ventajas y casos de uso más representativos:

- **Bases de datos relacionales en la nube (DBaaS)**

Las bases de datos relacionales como servicio (DBaaS) trasladan el modelo relacional clásico al entorno cloud manteniendo sus propiedades fundamentales, como transacciones ACID, SQL estándar, integridad referencial y soporte para procedimientos almacenados. Su principal ventaja es la automatización de la infraestructura: respaldos automáticos, parches de seguridad, monitoreo, replicación para alta disponibilidad y mecanismos de conmutación por falla forman parte integral del servicio.

Además, estos servicios permiten escalar recursos de forma flexible, ya sea aumentando la capacidad de la instancia o incorporando réplicas de lectura. Algunas soluciones avanzadas ofrecen incluso escalabilidad horizontal global con consistencia fuerte, lo que las hace adecuadas para aplicaciones críticas distribuidas a gran escala.

- **Bases de datos documentales**

Las bases de datos documentales almacenan la información en documentos JSON o BSON, sin requerir un esquema rígido, lo que permite manejar estructuras de datos flexibles y en constante evolución. Este enfoque resulta especialmente útil cuando los datos son jerárquicos o presentan variaciones frecuentes entre registros, reduciendo la necesidad de migraciones de esquema complejas.

Al agrupar datos relacionados dentro de un mismo documento, se simplifican ciertos patrones de consulta y se evitan operaciones costosas de unión entre tablas. Estas bases de datos ofrecen lenguajes de consulta expresivos, soporte para índices secundarios, búsquedas de texto completo y operaciones geoespaciales, siendo ampliamente utilizadas en aplicaciones web y móviles modernas.

- **Bases de datos clave-valor**

Las bases de datos clave-valor representan el modelo NoSQL más simple: cada valor se asocia a una clave única, permitiendo accesos extremadamente rápidos. Esta simplicidad las hace ideales para escenarios donde se prioriza el rendimiento de lectura y escritura por encima de la complejidad de las consultas, con latencias en el orden de microsegundos o milisegundos.

Son fundamentales en arquitecturas modernas para implementar cachés distribuidos, manejo de sesiones, contadores, configuraciones y otros casos donde el acceso directo por identificador es predominante. Algunas soluciones amplían este modelo básico incorporando estructuras de datos avanzadas e índices secundarios para soportar casos de uso más complejos.

- **Bases de datos de grafos**

Las bases de datos de grafos modelan la información como nodos y relaciones, permitiendo representar de forma natural dominios donde las conexiones entre entidades son clave. A diferencia del modelo relacional, las relaciones son elementos centrales del modelo y se recorren de manera eficiente, incluso cuando el grafo crece en tamaño y complejidad.

Este enfoque resulta especialmente adecuado para redes sociales, sistemas de recomendación, detección de fraude y análisis de dependencias, donde es necesario explorar relaciones profundas entre entidades. El uso de lenguajes de consulta especializados permite navegar el grafo de forma expresiva y con un costo computacional estable.

- **Bases de datos de serie temporal, en memoria y libro mayor**

Las bases de datos de serie temporal están optimizadas para manejar datos indexados por tiempo, aplicando técnicas especializadas de compresión y consulta para responder eficientemente a rangos temporales, agregaciones y ventanas deslizantes. Son ampliamente utilizadas en monitoreo, IoT, análisis financiero y telemetría.

Por su parte, las bases de datos en memoria priorizan el rendimiento al almacenar los datos directamente en RAM, siendo clave en sistemas de caché y aplicaciones de alta concurrencia. Finalmente, las bases de datos de libro mayor introducen mecanismos de inmutabilidad e integridad criptográfica, garantizando un historial verificable de cambios, lo que las hace especialmente valiosas en contextos financieros y regulados.

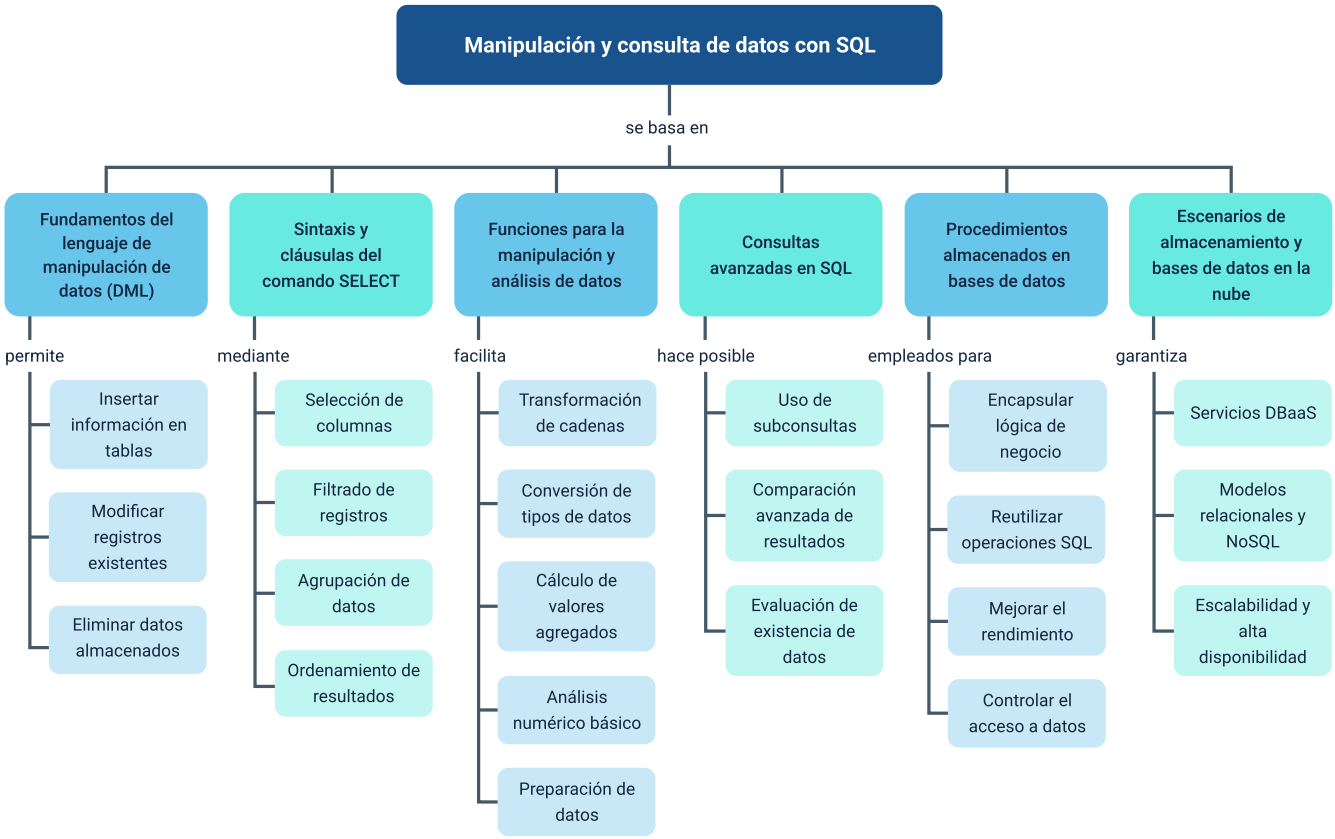
Para finalizar, se presenta la siguiente tabla, cuyo objetivo es integrar de forma sintética los modelos de bases de datos utilizados en entornos cloud, destacando sus fortalezas principales, las limitaciones más relevantes y los escenarios de uso para los que resultan más adecuados. Esta visión comparativa permite identificar rápidamente el modelo que mejor se ajusta a las necesidades técnicas y de negocio de un sistema de información:

Tabla 9. Síntesis de modelos de bases de datos en la nube y criterios de uso

Modelo de base de datos	Fortaleza principal	Limitación clave	Caso de uso recomendado
Relacional.	Transacciones ACID completas y SQL estándar.	Escalado horizontal complejo.	Sistemas transaccionales críticos (ERP, finanzas, contabilidad).
Documental.	Esquema flexible y datos jerárquicos.	JOINs y consultas relacionales limitadas.	Aplicaciones web/móviles, catálogos, perfiles de usuario.
Clave-Valor.	Latencia ultra-baja y acceso O(1).	Sin consultas complejas ni relaciones.	Caché, sesiones, contadores, configuraciones.
Grafo.	Navegación eficiente de relaciones profundas.	Ineficiente para consultas no relacionales.	Recomendaciones, fraude, redes sociales.
Serie temporal.	Optimizada para datos con timestamp.	Poco flexible para otros tipos de datos.	IoT, monitoreo, métricas y telemetría.
En memoria.	Rendimiento extremo (<1 ms).	Alto costo por GB y dependencia de RAM.	Caché distribuido y datos de alta concurrencia.
Libro mayor.	Inmutabilidad y auditoría verificable.	Modelo append-only, menor flexibilidad.	Auditoría regulatoria, finanzas, trazabilidad.

Síntesis

El recorrido formativo ha abordado de manera integrada los fundamentos técnicos y de gestión del desarrollo de software, articulando la calidad del código, las pruebas y la prevención de errores con la estructuración, mantenibilidad y aplicación de principios de diseño orientados a la evolución del sistema. De forma complementaria, se analizó el papel de los manuales de usuario y la redacción técnica como elementos clave para la adopción y su uso efectivo, enfatizando la claridad, la secuencialidad y la adecuación al perfil del usuario. Asimismo, se estudió la gestión del proyecto como un ciclo continuo de planificación, ejecución, medición y corrección, incorporando control, seguimiento, gestión de riesgos, comunicación y enfoques metodológicos. Finalmente, se profundizó en la naturaleza de los costos del software, su comportamiento no lineal, los distintos tipos de costos, la estimación y los mecanismos de ajuste ante desviaciones, proporcionando una visión integral que permite comprender el desarrollo de software como un proceso técnico, organizacional y económico orientado a la calidad y la viabilidad de proyectos reales.


Manipulación y consulta de datos con SQL
se basa en
Fundamentos del lenguaje de manipulación de datos (DML)
Sintaxis y cláusulas del comando SELECT
Funciones para la manipulación y análisis de datos
Consultas avanzadas en SQL
Procedimientos almacenados en bases de datos
Escenarios de almacenamiento y bases de datos en la nube
permite
Insertar información en tablas
Modificar registros existentes
Eliminar datos almacenados
mediante
Selección de columnas
Filtrado de registros
Agrupación de datos
Ordenamiento de resultados
facilita
Transformación de cadenas
Conversión de tipos de datos
Cálculo de valores agregados
Análisis numérico básico
Preparación de datos
hace posible
Uso de subconsultas
Comparación avanzada de resultados
Evaluación de existencia de datos
empleados para
Encapsular lógica de negocio
Reutilizar operaciones SQL
Mejorar el rendimiento
Controlar el acceso a datos
garantiza
Servicios DBaaS
Modelos relacionales y NoSQL
Escalabilidad y alta disponibilidad

Glosario

ACID: acrónimo de Atomicity, Consistency, Isolation, Durability. Conjunto de propiedades que garantizan que las transacciones de base de datos se procesan de manera confiable: o todas las operaciones de la transacción se completan (commit) o ninguna se aplica (rollback).

Agregación: operación SQL que calcula un único valor resumen (suma, promedio, conteo, máximo, mínimo) sobre un conjunto de filas mediante las funciones COUNT, SUM, AVG, MAX y MIN, usualmente en combinación con la cláusula GROUP BY para obtener resultados por grupos de datos.

Clave foránea (FK): columna o conjunto de columnas en una tabla que hace referencia a la clave primaria de otra tabla, estableciendo una relación de integridad referencial. Garantiza que no se puedan insertar registros que hagan referencia a valores inexistentes en la tabla referenciada.

Clave primaria (PK): columna o combinación de columnas que identifica de manera única cada fila de una tabla. No puede contener valores NULL ni duplicados y es la base sobre la cual se construyen las referencias de integridad referencial del esquema relacional.

DBaaS: Database as a Service. Modelo de servicio de computación en la nube donde el proveedor gestiona la infraestructura subyacente de la base de datos (hardware, sistema operativo, motor de BD, parches, respaldos) y el cliente solo administra los datos y las aplicaciones.

DML: Data Manipulation Language. Subconjunto del lenguaje SQL que incluye los comandos para manipular los datos almacenados en la base de datos: SELECT para consultar, INSERT para insertar, UPDATE para modificar y DELETE para eliminar registros.

EXISTS: operador SQL para subconsultas que devuelve TRUE si la subconsulta produce al menos una fila, sin importar los valores específicos. Más eficiente que IN para grandes conjuntos porque detiene la búsqueda en el primer match encontrado.

GROUP BY: cláusula del SELECT que agrupa las filas del resultado con valores iguales en las columnas especificadas, permitiendo aplicar funciones de agregación para obtener un único valor resumen por grupo en lugar de un valor por cada fila individual.

HAVING: cláusula del SELECT que filtra los grupos generados por GROUP BY basándose en condiciones que involucran funciones de agregación. Actúa después del agrupamiento, a diferencia de WHERE que actúa antes sobre filas individuales.

JOIN: operación SQL que combina filas de dos o más tablas basándose en una condición de enlace (generalmente igualdad entre clave foránea y clave primaria). Los tipos son: INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER y CROSS JOIN.

NoSQL: Not Only SQL. Término que agrupa los sistemas de gestión de bases de datos que no usan el modelo relacional ni el lenguaje SQL como principal mecanismo de consulta. Incluye modelos documentales, clave-valor, de grafo, de columnas anchas y de serie temporal.

NULL: valor especial en SQL que representa la ausencia de datos o un valor desconocido. No es igual a cero ni a una cadena vacía; su evaluación requiere los

operadores IS NULL e IS NOT NULL ya que NULL comparado con cualquier valor (incluyendo NULL) devuelve NULL.

Procedimiento almacenado: conjunto de sentencias SQL precompiladas y almacenadas en el servidor de base de datos, ejecutables mediante una llamada con nombre y parámetros. Mejoran el rendimiento (plan precompilado), la seguridad (encapsulamiento) y el mantenimiento del código de acceso a datos.

Subconsulta: sentencia SELECT anidada dentro de otra sentencia SQL (SELECT, INSERT, UPDATE o DELETE). Se clasifica según su posición (en WHERE, FROM o SELECT), el número de filas que devuelve (escalar, de fila o de tabla) y su dependencia de la consulta exterior (correlacionada o no correlacionada).

Transacción: unidad lógica de trabajo en la base de datos compuesta por una o más operaciones SQL que se ejecutan de manera atómica: o todas tienen éxito y se confirman (COMMIT) o todas se revierten (ROLLBACK), garantizando la integridad de los datos ante fallos del sistema.

Referencias bibliográficas

Coronel, C. & Morris, S. (2018). Sistemas de bases de datos: Diseño, implementación y administración (11.ª ed.). Cengage Learning.

Date, C. J. (2019). Database design and relational theory: Normal forms and all that jazz (2nd ed.). Apress.

Elmasri, R. & Navathe, S. (2016). Fundamentos de sistemas de bases de datos (7.ª ed.). Pearson Educación.

Forta, B. (2020). SQL in 10 minutes a day, Sams teach yourself (5th ed.). Sams Publishing.

Microsoft. (2024). Transact-SQL reference (Database Engine). Microsoft Learn. Microsoft Learn – T-SQL Reference.

MySQL. (2024). MySQL 8.0 reference manual. Oracle Corporation. MySQL 8.0 Reference Manual

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. PostgreSQL 16 Documentation.

Silberschatz, A., Korth, H. F. & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill Education.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Juan Daniel Polanco Muñoz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima